

Virtual End-to-End Encryption: Analysis of the Doctolib Protocol

Dennis Dayanikli
Hasso-Plattner-Institute,
University of Potsdam, Germany

Laura Holz
Hasso-Plattner-Institute,
University of Potsdam, Germany

Anja Lehmann
Hasso-Plattner-Institute,
University of Potsdam, Germany

Abstract

Doctolib is a popular healthcare platform, used by over 90 million users across France, Italy, and Germany. One of its main features is the secure data exchange between patients and doctors, with 7 million documents shared per month. Doctolib claims to provide the "world's first end-to-end encryption platform built for health applications". The encryption protocol, described in a Whitepaper and Github repository, relies on envelope encryption and lets users upload ciphertexts for the secure data exchange. The ciphertexts are stored and retrieved through a distributed system, consisting of a data server and a key server. To access the data, recipients fetch the ciphertexts and decrypt them with their private key. However, the platform does not require end-users to maintain any cryptographic keys themselves and instead relies on a *virtual device* that leverages the two-server setting. The virtual device splits the user's private key over both servers, and uses password-based authentication for its retrieval. Overall, the goal of the protocol is to ensure confidentiality of the uploaded medical records as long as at most one server is corrupt. In this work, we analyze the security of Doctolib's distributed encryption protocol. First, we define a set of formal security models for such *password-based distributed envelope encryption*, that capture the optimal security properties under different corruption settings. We then analyze the protocol – abstracted from the available information – in our model, and show that it does not achieve the desired security guarantees. We finally propose a simple modification that strengthens the original protocol through the use of a distributed oblivious pseudorandom function that provably achieves all our security properties.

CCS Concepts

• Security and privacy → Cryptography; Key management.

Keywords

E2EE, cloud storage, envelope encryption, password-based security

ACM Reference Format:

Dennis Dayanikli, Laura Holz, and Anja Lehmann. 2025. Virtual End-to-End Encryption: Analysis of the Doctolib Protocol. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*, August 25–29, 2025, Hanoi, Vietnam. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3708821.3710841>

Contact authors: dennis.dayanikli@hpi.de, anja.lehmann@hpi.de.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ASIA CCS '25, Hanoi, Vietnam*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1410-8/25/08

<https://doi.org/10.1145/3708821.3710841>

1 Introduction

With the advent of digitization influencing all aspects of our lives, including the way we manage our health, online platforms have become integral in facilitating tasks such as booking doctor's appointments. For this purpose, in France, Germany, and Italy, patients can use a platform provided by Doctolib, a software provider in the digital health sector. Apart from streamlining appointment scheduling for patients, Doctolib also enables telemedical video consultations, and facilitates communication and data exchange between doctors and patients. Founded in 2013, Doctolib is used by 900,000 healthcare professionals and 90 million patients across Europe [21], and was even commissioned by various governments to organize vaccine appointments during the COVID-19 pandemic [8, 23].

End users can access Doctolib's services either through a mobile app on their phones, or directly from the browser without the need of any pre-installed application. An important feature of the platform is the secure sharing of medical data between patients and health practitioners, with 7 million documents shared per month [21]. After activating the "My Documents" feature, users can upload medical documents and share them with other users, such as healthcare practitioners, using Doctolib's cloud service.

While convenient, this also introduces significant challenges related to data security and privacy. Doctolib guarantees data confidentiality through an encryption protocol developed by Tanker, an encryption startup they acquired in 2022 [18]. In a press release about this acquisition, Doctolib claimed that "Tanker has worked with Doctolib to co-create the world's first end-to-end encryption platform built for health applications" [20], and promised that "end-to-end encryption will continue to be implemented to all Doctolib solutions to secure personal health information" [20]. Tanker's cryptographic protocol is open-source and documented on GitHub [33] and its integration in Doctolib is described in a whitepaper [19].

Distributed Envelope Encryption & Key Management. The Tanker protocol used by Doctolib provides "end-to-end encryption as a service" [33]. In its core, the actual data encryption is based on the classical envelope encryption idea, encrypting the data symmetrically and wrapping the symmetric resource key under each public key of the recipients. Tanker then proposes a *two-server setting*, consisting of a data and key server, to split the storage of the encrypted data: the data server stores the encrypted files, whereas the key server stores the encrypted resource keys. When a user wants to access data shared with her, she fetches both and locally decrypts the resource key with her secret key (see Fig. 1).

Clearly, the security and usability strongly depends on how this user secret key is managed, and Tanker proposes a complex protocol to protect and access such a key. It introduces two additional layers of encryption: (1) A device-specific (asymmetric) encryption key is used to protect and share the user's secret key among the physical devices owned by the user. (2) A server-hosted (symmetric) key is used to encrypt the device-specific secret key, used to protect the

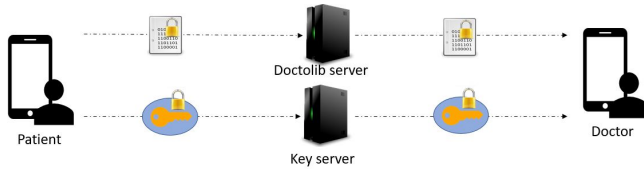


Figure 1: Overview of Doctolib’s infrastructure [19] for the distributed envelope encryption. The encrypted data is stored on the Doctolib server (our data server DS), whereas the encrypted resource key is kept on the key server KS.

device key when it is not in use. In addition, signature keys, again stored by the user’s device, and a Trustchain are used to adaptively enroll and revoke devices from accessing the user’s keys.

Usability vs. Security. Relying on cryptographic keys under full control of the user’s local devices can yield secure end-to-end encryption [29], but also comes with the inherent risk that users lose access to their data when they lose their physical device. In Doctolib, this is considered too much of a risk [30]:

“In a classic E2EE scheme, if a user loses his device (containing his private key), all his encrypted data may be lost. However, in the healthcare sector, data loss is an unacceptable risk. [...] Thus we implemented a second mechanism to recover the private key based on double authentication to ensure practitioners will always be able to decrypt their documents, even 10 years after encryption.”

To remedy this risk, Doctolib relies on the concept of *virtual devices*, that was also introduced by Tanker and uses the two-server setting to distribute the physical device key. The Doctolib Whitepaper [19] presents the virtual device as a mechanism to guarantee that users can access their data without having to handle cryptographic keys themselves. To this end, Doctolib also introduces the concept of “transparent double authentication” which requires that the user can bootstrap double, but “independent” authentication to both servers from a single password in order to retrieve her virtual device, and consequently access her encryption key. The goal is that the virtual device – and the user’s private key it protects – remains secure, even if one of the servers gets compromised.

E2EE: Theory vs. Reality. A crucial part of the Tanker protocol is its complex support of multi-device handling, and strong protection of the user’s encryption key, with the virtual device serving only as an *optional* key recovery feature [34]. The Doctolib adoption, as described in their Whitepaper [19], then states that a virtual device is created per *default* in addition to physical devices. And finally, the actual deployed Doctolib service seems to *solely* rely on the virtual device, as neither the mobile app nor browser application give any indication on how to register or revoke new device keys. Making the virtual device mandatory has clear advantages for usability, as it allows users to log in from any browser or new device without the need to manage keys.

However, by defaulting to the virtual device, or already making it a mandatory feature, Doctolib’s system essentially gives up on end-to-end security. Ideally, the distributed setting and split of all ciphertexts and keys over two servers, should still yield strong – *virtual-end-to-end* – security as long as at most one of the servers

gets compromised. To the best of our knowledge, no security analysis of this virtual-device based distributed envelope encryption by Doctolib has been done yet. Thus, the security of this widely deployed real-world protocol is an interesting open question.

1.1 Our Contributions

In this work, we analyze the security guarantees of Doctolib’s encryption protocol¹. Therefore, we first define a formal security model that captures the envelope encryption in the two-server setting, and enables users to encrypt and share data with other users, while bootstrapping the service from passwords only.

We call the resulting model password-based distributed envelope encryption (DEEP), based on Doctolib’s protocol in the virtual device setting. Our model enables users to register, encrypt messages for other users, and decrypt received messages, all while interacting with both servers.

We then set out the desired security guarantees of DEEP which cover the optimal achievable security guarantees with Doctolib’s usability considerations in mind.

Formalizing Security Guarantees. The main security goal is confidentiality: only the designated receivers of the encrypted data should be able to access the corresponding plaintext. We put forward two notions of confidentiality:

Strong Confidentiality models confidentiality in the presence of a strictly passive adversary, and guarantees that the ciphertexts stored on the servers do not leak any information about the data. In essence, if the secret shares to reconstruct the virtual device key are on completely “separate infrastructures [...] with no overlapping access rights” as claimed by Doctolib [19], then the servers learn nothing about the data from the ciphertexts, thus providing end-to-end encryption in this scenario.

Online Confidentiality considers an attacker who may actively participate in the protocol, either as a legitimate user or trying to impersonate an existing honest user. If the attacker manages to successfully authenticate towards the servers, he can recover the virtual device key and access the data. However, the “transparent double authentication” should still provide security by limiting the attacker to a single password guess per interaction. Furthermore, even if the attacker compromises one of the servers, they should still be restricted to one password guess per interaction. This models the requirement for transparent double authentication, that “at no time should the user provide information to the [data] server that could be used to authenticate with the key server, and vice versa.”[19].

In a setting where both servers are compromised, we do not model any confidentiality guarantee. Although our DEEP model could technically support such a guarantee, we omit it here since Doctolib requires that the servers must always be able to recover the users’ data. However, we aim to ensure that compromising both servers does not immediately expose user passwords, which may be reused across other services. This is covered as follows:

¹Disclaimer: Our analysis focus on the theoretical design of the protocol as described in Doctolib and Tanker’s public documentation, assuming that all building blocks are instantiated correctly. Our analysis did not study the concrete implementation or the deployed Doctolib application.

Precomputation Attack Resistance If both servers are compromised, the attacker can only recover a user password with offline attacks, which require computational effort for each guess and must be performed *after* both servers are compromised, preventing precomputation attacks using e.g. rainbow tables.

Analysis of Doctolib's Protocol π_{DEEP} . We then analyze whether Doctolib's encryption protocol π_{DEEP} satisfies these optimal security guarantees. We show that π_{DEEP} achieves strong confidentiality under standard assumptions on the encryption schemes, but does not achieve online confidentiality and precomputation attack resistance. The main problem of π_{DEEP} is its use of "independent password hashes" for authentication. These stored hashes allow an attacker who compromises one server to perform offline password attacks. The attacker can then use the recovered password to authenticate to the second server, obtain the user's virtual device and decrypt all her data.

Improved Protocol π_{DEEP}^+ . To address these issues, we propose an improved DEEP protocol, which provably realizes all the proposed security guarantees. This enhanced protocol retains the encryption structure of π_{DEEP} but replaces hash-based authentication with a distributed Oblivious Pseudorandom Function (dOPRF) approach. This change prevents a single server from storing sufficient information for offline attacks and requires an attacker to interact with both servers to conduct password tests. Apart from standard security properties, this protocol requires robustness from the underlying data encryption scheme. This property enforces a binding between a ciphertext and a key, and prevents an attacker from passing off a ciphertext to be valid under two different keys. Our solution has minimal computational overhead compared to π_{DEEP} while providing improved security guarantees.

1.2 Related Work

Virtual Device as PPSS. Password-protected secret sharing (PPSS) [6, 25–27] enables a user to share a secret among several servers and later retrieve it using a subset of them, provided the same password is used during sharing and retrieving. Doctolib's virtual device acts as a 2-out-of-2 PPSS, where a user can recover the virtual device's secret key if she provides the correct password to both servers. However, unlike PPSS, where corrupt servers still require offline attacks to reconstruct the secret, Doctolib's virtual device key can be immediately reconstructed by the servers to prevent data loss.

Password-Based Cloud Storage. Several works like OKMS [28] and DPASE [16] convert passwords with server assistance into encryption keys which can be used to encrypt data in a cloud storage protocol. Chen et al. [15] propose a distributed cloud storage solution where a key server helps derive encryption keys from passwords, and encrypted data is stored on a data server. In contrast, Doctolib decouples encryption keys from passwords. Otherwise, a brute-force attack on the password would be required to recover data. Furthermore, none of these works allow file sharing between users. This is due to their dependence on passwords, which would require sharing (file-specific) passwords to enable data sharing.

Formal Model of Cloud Storage. Backendal et al. [4] present a formal analysis of end-to-end encrypted cloud storage, covering

features such as file upload, download, sharing, and updating. Their work analyzes a single server setting with a focus on end-to-end confidentiality and integrity, modeling security in the presence of a corrupt server. Their work also considers password-based user authentication. They circumvent the issue of file-sharing in password-based schemes by allowing users to share unencrypted file keys via an out-of-band channel. In contrast to their work, our security model focuses on the confidentiality of the protocol in the distributed server setting and has two security guarantees which they do not cover: Security in the presence of a passive attacker where strong security guarantees are desired, and an external attacker trying to impersonate an honest user, modeling security provided by the password-based authentication method.

Analyzing Real-World Cloud Storage Protocols. Recently there has been a series of works showcasing attacks on real-world deployed cloud storage solutions such as MEGA [3, 5] and Nextcloud [2]. Most of these attacks exploit incorrect implementations of the cryptographic building blocks. In contrast, our work focuses on the security guarantees given by the formal building blocks.

1.3 Disclosure

We disclosed our findings to Doctolib's security team on December 16th 2024. We informed them that the protocol described in their whitepaper does not achieve the optimal security guarantees that are possible in such a distributed encryption protocol, specifically regarding security in the event of a server compromise. As of August 22nd 2025, we have not received a response yet. However, since our work primarily focuses on the formal analysis of the protocol described in the whitepaper and its potential improvements, rather than critical vulnerabilities in the deployed system, we do not believe the findings require immediate action from Doctolib.

2 Preliminaries

Notation. Throughout this paper, we use λ to denote the security parameter. For an algorithm A , $y \leftarrow A(x)$ signifies running A on input x to produce output y . With $r \leftarrow_{\mathcal{R}} S$ we represent that a value r is chosen uniformly at random from set S . We say that a function f is negligible if for any positive polynomial p , we have $f(\lambda) < 1/p(\lambda)$ for sufficiently large λ . For simplicity, we use negl to denote an unspecified negligible function.

Symmetric Encryption. SE consists of key generation $\text{KGen}(1^\lambda) \rightarrow k$, an encryption $\text{Enc}(k, m) \rightarrow c$, and decryption algorithm $\text{Dec}(k, c) \rightarrow m$. We require the encryption scheme to be indistinguishable against eavesdropping attacks (IND-EAV security), i.e. an attacker, given a ciphertext c of a *single* encryption, should not be able to derive any information about the underlying plaintext m .

Authenticated Encryption Scheme with Associated Data. AEAD consists of key generation $\text{KGen}(1^\lambda) \rightarrow k$, encryption $\text{Enc}(k, m, ad) \rightarrow c$, which encrypts message m using key k and associated data ad , and the corresponding decryption $\text{Dec}(k, c, ad) \rightarrow m$ for ciphertext c using key k and associated data ad .

The AEAD scheme apart from IND-EAV security needs to have *one-time ciphertext integrity*, meaning that an adversary, given a ciphertext under an unknown key k for (m, ad) cannot provide a different (c^*, ad^*) -pair that decrypts properly under k . An AEAD

scheme with both IND-EAV security and one-time ciphertext integrity, is called *1-AEAD secure*. This is the AEAD equivalent of 1-AE security defined in [11] and further defined in App. A. Additionally, the AEAD scheme needs to have *robustness* which ensures that an adversary, when given a ciphertext c of (m, ad) encrypted under an unknown key k , cannot find a key $k^* \neq k$ and ad^* such that c decrypts properly under k^* and ad^* . The definition is given in more detail in App. A.

Public-key Encryption. PKE consists of key generation $\text{KGen}(1^\lambda) \rightarrow (sk, pk)$, an encryption $\text{Enc}(pk, m) \rightarrow c$, and decryption algorithm $\text{Dec}(sk, c) \rightarrow m$. In our paper, we use standard IND-CPA and IND-CCA security of PKE which we recall in App. A.

Hash Function. H that efficiently maps an input $\{0, 1\}^*$ of arbitrary size to a string of fixed length λ . In our security proofs, we model the hash function as a random oracle (RO).

3 Overview of the Doctolib System

This section provides an overview of the encryption protocol, key management and distributed system design used by Doctolib. From the core concepts presented in this section, we abstract our generic model of *password-based distributed envelope encryption* (DEEP), and later analyze the concrete Doctolib protocol π_{DEEP} in this model.

3.1 Setup & Functionality

The Doctolib app enables the exchange of data among application users through an encrypted cloud storage protocol. The high-level idea of this protocol is described in Doctolib’s whitepaper [19] and sketched in Fig. 1. It internally uses the Tanker encryption protocol, which is available on Github [33]. While most parts of the protocol are well-documented in the above sources, some aspects leave room for interpretation or are not specified at all. In these cases, we had to guess how the protocol could look like, and will make that clear throughout our discussion.

Entities & Functionality. There are two main entities involved in the encryption protocol: a set of *users* $U_1, \dots, U_n$ and the *Doctolib system*. A user aims to securely store and retrieve data via the Doctolib system, as well as share data with other users. In the medical context where the app is used, users are typically patients or practitioners. The core functionality is as follows:

Registration: To use the app, a user U first registers with the Doctolib system. This sets the authentication information and keys that later allows U to upload, share or retrieve data.

Encryption (Data Upload): A previously registered and properly authenticated user can upload data to the Doctolib system, where the uploaded data is encrypted for designated recipients (including the user herself).

Decryption (Data Retrieval): A user, again after proper registration and authentication, can obtain – and locally decrypt – selected data from the Doctolib system.

Security. The Doctolib whitepaper [19] states that their solution is “based on the known principles of end-to-end encryption, adapted to our mass-market use case”. It lists three security rules that shaped the design of their system, taken verbatim from [19]:

- (1) *It must be impossible for any Doctolib employee to access personal health information. Even if someone gained access to all user data stored by Doctolib, they should not be able to read any protected health information.*
- (2) *It must be impossible for any Doctolib employee to access encryption files. Encryption keys should be accessible only by end-users, and no one should be able to modify them.*
- (3) *End-user devices are assumed safe when in use. When not in use, no health data or encryption keys should be accessible.*

Distributed System. To achieve these goals, the Doctolib system is built in a distributed manner, splitting the overall system into a *data server*² (DS) and a *key server* (KS). Thus, the procedures to register, upload and download data with the Doctolib system are realized through interactive protocols that the user runs with both the DS and KS.

The core idea is that the uploaded data is split among the DS and KS, such that neither alone can access the user’s files. This is what we abstract as *distributed envelope encryption*. The second purpose of the distributed setup is to realize a so-called *virtual device*, which manages the user’s access to her long-term cryptographic key. We first explain the distributed envelope encryption, and then sketch the key management options envisioned by Tanker and Doctolib.

3.2 Distributed Envelope Encryption

Before uploading a new file to the Doctolib system, the user encrypts the data under a fresh symmetric key – the **resource key** rk . Further, the user encrypts the resource key under the public keys of the intended recipients. This requires all users to have **long-term user key pairs** (usk, upk) for an asymmetric encryption scheme. These keys are generated upon registration, and the public keys are available through the system.

This concept of using symmetric encryption for the file, and wrapping the key under the public keys of all recipients is known as *envelope encryption* and a widely-used method for multi-recipient data sharing [24].

An essential design decision of the Doctolib system is to rely on a *two-server setting* for the distributed storage of such envelope encrypted data: the files encrypted under the resource key are stored by the data server, whereas the encrypted resource keys are stored by the key server.

3.3 Key Management for End Users

Clearly, the security of the encrypted data relies on the secrecy of the user’s long-term secret key usk . The Tanker/Doctolib system proposes an elaborate key structure to access and protect that key (see Table 1 for an overview of the keys involved).

Key Wrapping. First, the user does not directly keep her long-term key herself. Instead, usk gets wrapped under a **device key pair** (dsk, dpk) . The idea is that users might want to access their data, for which they need usk , from multiple devices. To this end, a device-specific key pair (dsk, dpk) is created whenever the user registers a new device, e.g., a smartphone or laptop. The user then encrypts usk under dpk and stores c_{usk} with the key server KS.

²In the Tanker documentation, the DS is called the Application Server, and Doctolib calls it the Doctolib Server.

Keys	Usage
Resource Key rk	to encrypt data m : $c_D = \text{Enc}(rk, m)$
User Key Pair (usk, upk)	to encrypt rk : $c_K = \text{Enc}(upk, rk)$
Device Key Pair (dsk, dpk)	to encrypt usk : $c_{usk} = \text{Enc}(dpk, usk)$
User Secret uk	to encrypt dsk : $c_{dsk} = \text{Enc}(uk, dsk)$

Table 1: Overview of keys used in Doctolib/Tanker

When the user wants to retrieve data, the c_{usk} is returned from KS, and the user can use the device’s secret key dsk to obtain her usk and decrypt the encrypted data.

The dsk is kept on the user’s device – but again only in protected form. The goal is to strictly restrict access to that key, i.e., the key should only be accessible on the device when the app is actively in use, as expressed as security property (3). To this end, dsk gets encrypted under a symmetric key, called the **user secret** uk . The device stores the encrypted key, denoted as c_{dsk} , whereas uk is stored by the data server. The user’s device has to fetch this uk whenever the user wants to decrypt her data, which then unlocks access to dsk and usk .

Virtual Devices. Relying on user-controlled physical devices that can keep cryptographic key material is essential for achieving end-to-end secure encryption, and immediately ensures security properties (1) and (2). However, it also creates the challenge that users lose access to their data when they lose or switch their device – which is considered too much of a risk in the context of Doctolib’s health care context. To address this, Doctolib/Tanker introduce the concept of a *virtual device* which emulates the physical device, but does not require any permanent user-side storage of cryptographic values. Instead, it uses the two server setting to distribute the key material, such that no server alone can access the user’s keys. This is done by storing the encrypted device secret key c_{dsk} on the key server, instead of the physical device; while the user secret uk (under which c_{dsk} is encrypted) is again held by the data server.

Transparent Double Authentication. With the virtual device keys, which protect the user’s encryption key, being distributed over KS and DS, security of the encrypted data then fully hinges on the authentication of the user towards both servers. As the goal of virtual devices is to *not* require the user to maintain any key material herself, user authentication then relies on human-memorizable secrets only, i.e., passwords. The Doctolib whitepaper specifies two crucial design principles for this password-based authentication, which they refer to as *transparent double authentication*. First, the user’s authentication to both servers must be independent: “This means, that at no time should the user provide information to the main Doctolib sever (DS) that could be used to authenticate with the key server and vice versa”. Second, the independent authentications should be “completely transparent for the end-user”, meaning that the double authentication must be derived from a single password.

3.4 Security vs. Usability

Neither Tanker nor Doctolib enforce the existence of physical devices, which would be the only option to obtain proper end-to-end security. The Doctolib Whitepaper even states that a virtual device is created first per default and kept permanently, to ensure that a user can never lose access to her data. Thus, even when a physical device is used, such a virtual device is available as alternative

access option. In fact, the deployed Doctolib app never mentions any actual device key creation and the user can also access her data purely from a browser.

Doctolib protects the virtual device through a distributed setting, and aims to guarantee the security properties (1) and (2) – assuming that the two servers are properly separated and cannot be corrupted together, and the transparent double authentication fulfills the claimed security properties. However, even in that case, a virtual device clearly provides significantly weaker security than a physical one. As the security of a system is only as strong as its weakest link, and given that the virtual device is the default (or even only) option for users’ key management in the deployed Doctolib system, we focus solely on the virtual device setting in the rest of this work.

Omitted/Simplified Features. Given our focus on the virtual device, we also omit the multi-device handling supported by the Tanker protocol. Therein, the user can adaptively register and revoke physical devices. This requires further signature keys, cross-signing from previous devices and the use of a Trustchain, which is a complex protocol in itself. The only functionality we need from their Trustchain protocol is the secure storage of the wrapped user key ciphertexts, which we simply model as a database managed by the key server.

4 Security Model of DEEP

We now present our model of *password-based distributed envelope encryption* (DEEP), which we abstracted from Doctolib’s System described in Sec. 3. Our model accounts for the system’s distribution into a data server DS and key server KS, as well as the requirement that the encryption service must be accessible using only a single user password. This reflects the design choice of imposing the virtual device as the default mechanism.

We start by defining a generic syntax for DEEP, which then allows us to formulate the optimal security guarantees such a system should possess. In our security model, we focus on the confidentiality of the uploaded data and specify two core secrecy properties: *strong* and *online confidentiality* depending on the allowed corruption capabilities of the adversary. Online confidentiality captures the optimal security guarantees as long as one server is honest. For the setting where both servers are corrupt, our model does not guarantee any confidentiality anymore (as inherently implied by the usability requirement of Doctolib), but still ensures some protection of the user password. This property, which we term *precomputation attack resistance*, accounts for tendency of users to re-use passwords across services, and ensures that passwords are not exposed immediately even when both servers are corrupt.

4.1 Generic Syntax of DEEP

Our generic syntax follows the main functionality presented in Section 3.1, and allows a user with username uid and password pw to register with a system represented through a data server DS and key server KS. Subsequently, the user can re-authenticate under the previously established password to up- and download encrypted data. All functionalities are modelled as interactive protocols that the user runs with both servers.

While Doctolib’s app allows the user to authenticate once per *session* and perform multiple encryption or decryption requests,

Registration:	$\langle \text{RegDS}(uid), \text{RegU}(uid, pw), \text{RegKS}(uid) \rangle \rightarrow (vf_{DS}, upk; (vf_{KS}, upk))$ Registers the user identified by uid with password pw , resulting in verification tuples vf_{DS} , vf_{KS} , and user public key upk .
Encryption:	$\langle \text{EncDS}(uid, vf_{DS}), \text{EncU}(uid, pw, \{uid_i\}_{i=1}^n, m), \text{EncKS}(uid, vf_{KS}, DB_U) \rangle \rightarrow (\text{FAIL}/(rid, c_D); \text{FAIL}/rid; \text{FAIL}/(rid, \{uid_i, c_{K,i}\}_{i=1}^n))$ Encrypts message m from uid for recipients $\{uid_i\}_{i=1}^n$ if pw is correct, resulting in ciphertexts c_D , c_K , and resource identifier rid . If authentication fails, all parties output FAIL.
Decryption:	$\langle \text{DecDS}(uid, rid, vf_{DS}, DB_D), \text{DecU}(uid, pw, rid), \text{DecKS}(uid, rid, vf_{KS}, DB_K) \rangle \rightarrow (\text{FAIL}/\text{SUCC}; \text{FAIL}/m; \text{FAIL}/\text{SUCC})$ User uid can retrieve the message m associated with rid , if the pw is correct. Otherwise all parties output FAIL.

Figure 2: Interactive DEEP protocols to register, and enc/decrypt data with a data and key server DS, KS, from a password pw .

we opted for a simpler model where each request requires separate authentication. This approach avoids the complexity of modeling authenticated sessions which would significantly complicate our abstraction and security model. Importantly, this does not weaken the overall system's security, as any attacks feasible in our model would also apply to the real system, where users may always perform a single encryption or decryption request per login.

Definition 4.1 (DEEP). A DEEP system Γ consists of the interactive algorithms for registration, encryption and decryption specified in Fig. 2, and described below. We assume that all parties implicitly receive the security parameter 1^λ as input, and that the servers maintain databases: The DS holds DB_D to store encrypted messages, and the KS holds DB_K to store the associated keys, and DB_U to store the public keys of users.

Registration: A user registers with a unique user name uid and a password pw through an interactive protocol with the DS and KS. This process generates a public key upk to enable encryption towards that user, along with verification information for the user's re-authentication: vf_{DS} , handled by the DS, and vf_{KS} , handled by the KS. We assume the key server maintains a database of all registered public keys DB_U to which it adds upk : $DB_U := DB_U \cup \{uid, upk\}$.

Encryption: A previously registered user can trigger the encrypted upload of a message m for other registered users by providing her username uid , password pw , the message m , and a list of recipients $\{uid_i\}_{i=1}^n$. This list may be of arbitrary size and also include uid .

The two servers check the user's identity using the verification information generated at registration, and output FAIL if verification fails. Upon successful verification, the encryption of m for all recipients in $\{uid_i\}_{i=1}^n$ is generated and identified through a unique resource identifier rid . This rid is shared with all parties. To model the envelope encryption of the ciphertext, we denote the encryption of m as $(c_D, c_{K,1}, \dots, c_{K,n})$. The ciphertext c_D contains the encryption of the data and gets returned to DS who (implicitly) stores it in the database DB_D . The ciphertexts $(c_{K,1}, \dots, c_{K,n})$ contain the wrapped keys to access the data, where $c_{K,i}$ belongs to user uid_i . KS obtains these ciphertexts together with the corresponding uid_i and stores them in database DB_K .

Decryption: To access a resource, the user initiates the decryption protocol by providing her username uid , password pw , and the resource identifier rid as inputs. We assume that the servers enforce access control at the application layer and verify whether the user uid has the right to access the resource rid . Thus, in our model, the user can only query the decryption protocol for resources rid that have been explicitly shared with them. Both servers get uid, rid as input as well as their respective verification information and databases. If authentication fails, the user receives FAIL. Otherwise,

the user receives the plaintext data m . Additionally, the servers output a bit FAIL/SUCC indicating the success of user authentication, which allows tracking unsuccessful password guesses.

Correctness. A DEEP is correct if all honestly encrypted messages can be recovered by the designated recipients upon successful authentication. The formal correctness definition is given in App. C.

4.2 Security Model

The main security goal for a system that encrypts data shared between users is data confidentiality, i.e., only authorized parties can read the data.

Confidentiality. We distinguish between two notions of confidentiality: *Strong confidentiality* guarantees that ciphertexts are secure against passive eavesdroppers. The restriction to passive adversaries permits security that is independent of the user's password and as strong as "proper" encryption. Otherwise, the password dictionary size and an adversary's success capability in correctly guessing an honest user's password is an inherent limitation and captured in our *online confidentiality* notion. In this model, an attacker can actively participate in the protocol, and compromise one of the servers.

Both models formulate confidentiality in the classical notion of indistinguishability of ciphertexts. This assumes that the envelope encrypted (and distributedly stored) ciphertexts are available to the adversary right away. While this may seem overly generous, we believe it is a sensible choice. First, in the concrete Doctolib protocol (presented in Sec.5.1), the ciphertexts are generated on the user device, and sent to both servers. Our model captures that these ciphertexts could be leaked through potential malware on the user's device. Second, also the storage of the ciphertexts by both servers might be prone to leakage, e.g., through deployed data duplication or reduced system-side protection, since the data are "just ciphertexts". Overall, we did not want to make any confidentiality assumptions on these ciphertexts, and thus assume them to be known to the adversary.

Security against Full Corruption. Our confidentiality notions assume that at most one server is compromised, i.e., we do not guarantee any secrecy if both servers are corrupt. While such a stronger notion would be possible in theory, this was not a design goal from Doctolib (as usability is considered more important), and thus omitted in our model. What we still want to guarantee is (some) secrecy of the user password in such a full breach scenario, which we define as *precomputation attack resistance*.

Out of Scope: Integrity. Data integrity, i.e., ensuring that the data cannot be altered, is not covered by our work: we assume servers to be at most honest-but-curious, i.e., they cannot manipulate any

Experiment $\text{Conf}_{\mathcal{A},\Gamma}^x(1^\lambda)$ for $x \in \{\text{strong}, \text{online}\}$ $(DB_U, DB_D, DB_K) \leftarrow \text{Setup}(1^\lambda); \quad \text{triv} \leftarrow 0; \text{corrDS} \leftarrow 0; \text{corrKS} \leftarrow 0$ $HU \leftarrow \emptyset; CU \leftarrow \emptyset$ $(uid, \{uid_i^*\}_{i=1}^n, m_0, m_1, st_0) \leftarrow \mathcal{A}^{\text{oracles}}(1^\lambda)$ if $uid \notin HU \vee \exists i \in \{1, \dots, n\} : uid_i^* \notin HU \vee m_0 \neq m_1$ abort retrieve $(uid, pw, v_{fDS}, v_{fKS})$ from HU $b \leftarrow_{\mathcal{R}} \{0, 1\}$ $((rid^*, c_D^*); rid^*; (rid^*, \{uid_i^*, c_{K,i}^*\}_{i=1}^n), sst) \leftarrow \langle \text{EncDS}(uid, v_{fDS}),$ $\quad \text{EncU}(uid, pw, \{uid_i^*\}_{i=1}^n, m_b), \text{EncKS}(uid, v_{fKS}, DB_U) \rangle$ $DB_D := DB_D \cup \{(rid^*, c_D^*)\}; DB_K := DB_K \cup \{(\{rid^*, uid_i^*, c_{K,i}^*\}_{i=1}^n)$ $b' \leftarrow \mathcal{A}^{\text{oracles}}(rid^*, c_D^*, c_{K,1}^*, \dots, c_{K,n}^*, sst, st_0)$ return 1 if $(b' = b \wedge \text{triv} = 0 \wedge (\text{corrDS} = 0 \vee \text{corrKS} = 0))$ else return 0 $O_{H\text{Reg}}(uid)$ // honest registration if $\exists (uid, \cdot) \in DB_U$ abort; else $pw \leftarrow_{\mathcal{R}} \mathcal{D}_{pw}$ $(v_{fDS}; upk; (v_{fKS}, upk), sst) \leftarrow \langle \text{RegDS}(), \text{RegU}(), \text{RegKS}() \rangle$ $DB_U := DB_U \cup \{(uid, upk)\}$ $HU := HU \cup \{(uid, pw, v_{fDS}, v_{fKS})\}$ return (upk, regout) $O_{H\text{Enc}}(uid, \{uid_i\}_{i=1}^n, m)$ // honest encryption retrieve $(uid, pw, v_{fDS}, v_{fKS})$ from HU $((rid, c_D); rid; (rid, \{uid_i, c_{K,i}\}_{i=1}^n), sst) \leftarrow \langle \text{EncDS}(), \text{EncU}(), \text{EncKS}() \rangle$ $DB_D := DB_D \cup \{(rid, c_D)\}; DB_K := DB_K \cup \{(rid, uid_i, c_{K,i})\}_{i=1}^n$ return $(rid, c_D, c_{K,1}, \dots, c_{K,n}, sst)$ $O_{H\text{Dec}}(uid, rid)$ // honest decryption retrieve $(uid, pw, v_{fDS}, v_{fKS})$ from HU $((\text{succ}; m; \text{succ}), sst) \leftarrow \langle \text{DecDS}(), \text{DecU}(), \text{DecKS}() \rangle$ if $rid = rid^* \wedge \exists i \in \{1, \dots, n\} : uid = uid_i^*$ set $\text{triv} \leftarrow 1$ return (m, sst)	$O_{C\text{Reg}}(uid)$ // corrupt registration if $\exists (uid, \cdot) \in DB_U$ abort $(v_{fDS}; \cdot; (v_{fKS}, upk), sst) \leftarrow \langle \text{RegDS}(uid), \mathcal{A}(\cdot), \text{RegKS}(uid) \rangle$ $DB_U := DB_U \cup \{(uid, upk)\}; CU := CU \cup \{(uid, \perp, v_{fDS}, v_{fKS})\}$ return regout $O_{C\text{Enc}}(uid)$ // corrupt encryption retrieve $(uid, \cdot, v_{fDS}, v_{fKS})$ from HU $\cup$ CU $((\text{FAIL}/(rid, c_D); \cdot; \text{FAIL}/(rid, \{uid_i, c_{K,i}\}_{i=1}^n), sst) \leftarrow \langle \text{EncDS}(uid, v_{fDS}), \mathcal{A}(\cdot),$ $\quad \text{EncKS}(uid, v_{fKS}, DB_U) \rangle$ abort if DS or KS output FAIL $DB_D := DB_D \cup \{(rid, c_D)\}; DB_K := DB_K \cup \{(rid, uid_i, c_{K,i})\}_{i=1}^n$ return $(rid, c_D, c_{K,1}, \dots, c_{K,n}, sst)$ $O_{C\text{Dec}}(uid, rid)$ // corrupt decryption retrieve $(uid, \cdot, v_{fDS}, v_{fKS})$ from HU $\cup$ CU $((\text{FAIL}/\text{SUCC}; \cdot; \text{FAIL}/\text{SUCC}), sst) \leftarrow \langle \text{DecDS}(uid, rid, v_{fDS}, DB_D), \mathcal{A}(\cdot),$ $\quad \text{DecKS}(uid, rid, v_{fKS}, DB_K) \rangle$ return sst $O_{C\text{Corr}}(x)$ // server compromise if $x = \text{"DS"} : \text{set } \text{corrDS} \leftarrow 1$ return $\{(uid, v_{fDS}) : (uid, \cdot, v_{fDS}, \cdot) \in HU \cup CU\}$ if $x = \text{"KS"} : \text{set } \text{corrKS} \leftarrow 1$ return $\{(uid, v_{fKS}) : (uid, \cdot, \cdot, v_{fKS}) \in HU \cup CU\}$ else return $\perp$ With oracles, sst and regout defined as <table> <tr> <th>x</th> <th>oracles</th> <th>sst</th> <th>regout</th> </tr> <tr> <td>strong</td> <td>$O_{H\text{Reg}}, O_{H\text{Dec}}, O_{H\text{Enc}}$</td> <td>$\perp$</td> <td>$pw$</td> </tr> <tr> <td>online</td> <td>$O_{H\text{Reg}}, O_{H\text{Enc}}, O_{H\text{Dec}},$ $O_{C\text{Reg}}, O_{C\text{Enc}}, O_{C\text{Dec}}$</td> <td>if $\text{corrDS} = 1 : \pi_{DS}$ if $\text{corrKS} = 1 : \pi_{KS}$ else $\perp$</td> <td>if $\text{corrKS} = 1 : v_{fKS}, \pi_{KS}$ if $\text{corrDS} = 1 : v_{fDS}, \pi_{DS}$ else $\perp$</td> </tr> </table>	x	oracles	sst	regout	strong	$O_{H\text{Reg}}, O_{H\text{Dec}}, O_{H\text{Enc}}$	$\perp$	pw	online	$O_{H\text{Reg}}, O_{H\text{Enc}}, O_{H\text{Dec}},$ $O_{C\text{Reg}}, O_{C\text{Enc}}, O_{C\text{Dec}}$	if $\text{corrDS} = 1 : \pi_{DS}$ if $\text{corrKS} = 1 : \pi_{KS}$ else $\perp$	if $\text{corrKS} = 1 : v_{fKS}, \pi_{KS}$ if $\text{corrDS} = 1 : v_{fDS}, \pi_{DS}$ else $\perp$
x	oracles	sst	regout										
strong	$O_{H\text{Reg}}, O_{H\text{Dec}}, O_{H\text{Enc}}$	$\perp$	pw										
online	$O_{H\text{Reg}}, O_{H\text{Enc}}, O_{H\text{Dec}},$ $O_{C\text{Reg}}, O_{C\text{Enc}}, O_{C\text{Dec}}$	if $\text{corrDS} = 1 : \pi_{DS}$ if $\text{corrKS} = 1 : \pi_{KS}$ else $\perp$	if $\text{corrKS} = 1 : v_{fKS}, \pi_{KS}$ if $\text{corrDS} = 1 : v_{fDS}, \pi_{DS}$ else $\perp$										

Figure 3: Confidentiality experiment for DEEP for strong and online confidentiality. To increase readability, we omit the inputs to the interactive algorithms in the honest oracles, as they should be clear from the context.

of the stored data. However, ciphertexts are uploaded and shared by users, which can be fully corrupt, and we ensure that these capabilities do not interfere with the expected confidentiality goals. A full-fledged security model for the desired integrity features is left as interesting future work.

4.2.1 Strong Confidentiality. If both servers are honest and the attacker is fully passive (i.e., does not even interact with the system as a corrupt user), we expect the strongest guarantee that is entirely independent of the user's passwords and ensures the same level of security as "normally" encrypted data. Strong confidentiality guarantees that ciphertexts in DEEP reveal no information about the underlying plaintext to a passive eavesdropper.

Our definition, depicted in Fig. 3, is captured as a left-or-right indistinguishability experiment (similar to classical IND-CCA security for encryption) played between a challenger and an adversary $\mathcal{A}$. It requires the adversary to provide two messages (m_0, m_1) together with a sending user uid and receiving users $\{uid_i^*\}_{i=1}^n$, upon which it receives the ciphertexts $c_D^*, c_{K,1}^*, \dots, c_{K,n}^*$ created through the interactive encryption protocol for message m_b for a random bit b . The adversary must determine b better than by guessing.

Throughout the experiment, the adversary can eavesdrop on the protocol executed between honest users and servers via oracles $O_{H\text{Reg}}, O_{H\text{Dec}},$ and $O_{H\text{Enc}},$ triggering the respective interactive

algorithms. $O_{H\text{Reg}}$ allows the adversary to register honest parties initialized with a random password pw . Since the adversary cannot actively engage as a user, strong confidentiality should not depend on the password itself; hence, pw is disclosed to the adversary after registration. A stronger notion would let the adversary choose the password, but we avoid this to maintain compatibility with the online confidentiality experiment. The oracle $O_{H\text{Enc}}$ lets $\mathcal{A}$ encrypt arbitrary data on behalf of a registered user uid towards other registered users $\{uid_i\}_{i=1}^n$, with the adversary learning the outputs $(rid, c_D, c_{K,1}, \dots, c_{K,n})$ to model the potential leakage of ciphertexts. The oracle $O_{H\text{Dec}}$ lets the adversary decrypt a valid resource rid for a registered user uid and learn the message m .

The adversary wins as long as it does not query $O_{H\text{Dec}}$ on the challenge rid^* for one of the honest recipients uid_i^* which would yield a trivial win.

Definition 4.2 (Strong Confidentiality). A DEEP is called strongly confidential, if for all PPT adversaries $\mathcal{A}$: $\Pr[\text{Conf}_{\mathcal{A},\Gamma}^{\text{strong}}(1^\lambda)] \leq 1/2 + \text{negl}$.

4.2.2 Online Confidentiality. In DEEP, the user authentication towards both servers is purely password-based. Thus, once the adversary becomes active, it can try to impersonate an honest user by making guesses against the password. Consequently, the security

then depends on the strength of the password. We want DEEP to provide confidentiality of the honest user's data, unless the attacker correctly guessed the user's password. Importantly, the attacker should only be able to test their password through interaction with both servers. This type of attack, known as an *online* attack, can be detected by an honest server and be mitigated (which sets the attack apart from *offline* attacks). This limitation to online attacks should still hold when the adversary corrupts one of the servers – as expressed through the independent authentication goal in the Doctolib whitepaper [19]. Therefore, we allow the adversary to corrupt one server in this model.

The online confidentiality experiment (Fig. 3) builds upon the strong confidentiality experiment. The adversary's task still remains to decide which message m_0 or m_1 was encrypted from a sender uid^* to recipients $\{uid_i^*\}_{i=1}^n$. However, next to the eavesdropping oracles, the adversary now also gets access to oracles allowing it to register corrupt users (O_{CReg}), actively participate in encryption and decryption protocols on behalf of corrupt and honest(!) users (O_{CDec} , O_{CEnc}), and compromising one of the servers (O_{Corr}).

The oracles (O_{CReg} , O_{CDec} , O_{CEnc}) allow the adversary to participate in the interactive protocols as a user, which can either be a user that was corrupt from the beginning, or trying to impersonate an honest user. In this user-role, the adversary can deviate arbitrarily from the protocol. The server-side part of the protocols are run honestly by the challenger. However, if the adversary has corrupted one of the servers, the oracles also return the compromised server's full view and internal state (which we denote as π_{DS} or π_{KS}).

For the adversary's win condition, we need two conditions to exclude trivial or inherent wins: (1) all recipients of the challenge message must be honest users, and (2) since users are only authenticated based on a password of potentially low entropy, there is a non-negligible chance that the adversary guesses the password of a recipient correctly and can decrypt the ciphertext. Since each query to O_{CEnc} and O_{CDec} reveals whether the password is correct, they effectively serve as a password test oracle. We thus denote all queries to O_{CEnc} and O_{CDec} for one of the challenge recipients' uid 's as q , and require that the attacker's success probability is not significantly bigger than $1/2 + q/|\mathcal{D}_{pw}|$.

Definition 4.3 (Online Confidentiality). A DEEP is called online confidential if for all PPT adversaries $\mathcal{A}$: $\Pr[\text{Conf}_{\mathcal{A},\Gamma}^x(1^\lambda)] \leq 1/2 + q/|\mathcal{D}_{pw}| + \text{negl}$ where $\mathcal{D}_{pw}$ is the set of all possible passwords, and q is the number of queries to O_{CEnc} or O_{CDec} for all challenge recipients' uid 's.

No E2E Security. In end-to-end (E2E) secure schemes [29], only the sending and receiving users should have access to the plaintext data, ensuring that servers cannot learn anything about the data. However, this requires the use of (strong) user-controlled cryptographic keys. In contrast, DEEP relies on users remembering a password of potentially low entropy. Consequently, DEEP does not have E2E security, as the two servers could always emulate the decryption protocol on behalf of a user and have a non-negligible probability of recovering the plaintext by correctly guessing the user's password. In fact, in the medical context where DEEP is employed, losing access to data can have severe consequences. Therefore, the system ensures data recoverability by the two servers without needing to perform such an offline attack. This favors usability over

Experiment $\text{PAR}_{\mathcal{A},\Gamma}(1^\lambda)$

```

 $(DB_U, DB_D, DB_K) \leftarrow \text{Setup}(1^\lambda); \text{corrDS} \leftarrow 0, \text{corrKS} \leftarrow 0, q_{\text{off}} \leftarrow 0$ 
 $uid \leftarrow_{\mathcal{R}} \mathcal{D}_{uid}, pw \leftarrow_{\mathcal{R}} \mathcal{D}_{pw}$ 
 $(vf_{DS}, upk; (vf_{KS}, upk), \pi_{DS}, \pi_{KS}) \leftarrow (\text{RegDS}(uid), \text{RegU}(uid, pw), \text{RegKS}(uid))$ 
 $pw^* \leftarrow \mathcal{A}^{O_{Corr}, O_{TestPwD}}(uid, upk)$ 
return 1 if  $pw^* = pw$ ; else return 0

 $O_{Corr}(x)$ 
if  $x = \text{"DS"}$  return  $(vf_{DS}, \pi_{KS})$ , set  $\text{corrDS} \leftarrow 1$ 
if  $x = \text{"KS"}$  return  $(vf_{KS}, \pi_{KS})$ , set  $\text{corrKS} \leftarrow 1$ 
else return  $\perp$ 

 $O_{TestPwD}(pw')$ 
if  $(\text{corrDS} = 1 \wedge \text{corrKS} = 1) : \text{set } q_{\text{off}} \leftarrow q_{\text{off}} + 1$  and return 1 if  $pw = pw'$ 

```

Figure 4: Security Experiment $\text{PAR}_{\mathcal{A},\Gamma}(1^\lambda)$

security but as a result, DEEP does not provide confidentiality in a fully-compromised setting where both servers collude.

4.2.3 Precomputation Attack Resistance. While we do not guarantee confidentiality for the encrypted data when both servers are corrupt, this usability tradeoff should not affect the secrecy of the user's password. That is, even when both servers are corrupt, the adversary should not be able to learn the passwords immediately. Instead the adversary must perform an offline attack on each password file. Additionally, all computations in such an offline attack must occur *after* both servers are compromised, preventing precomputation attacks using methods like rainbow tables.

In our security experiment (Fig. 4), the game initializes a challenge user with a randomly chosen password from the password dictionary $\mathcal{D}_{pw}$, and lets the adversary adaptively corrupt both servers independently through oracle O_{Corr} . The adversary wins if he can recover the user's password. The O_{Corr} oracle leaks the user's verification tuples stored on the server for re-authentication, as well as the transcript of the registration protocol.

The best attack strategy for an attacker should be offline attacks which should only be possible after compromising both servers. To monitor these offline attacks, we follow the approach used in other password-based protocols [12, 13, 16, 17], and grant $\mathcal{A}$ a $O_{TestPwD}$ oracle, which returns 1 if the guessed password is correct and tracks these offline attacks as q_{off} . Consequently, a security proof for this property therefore inherently requires an idealized assumption, such as the random oracle model.

Note that we did not model that the attacker engages actively on behalf of the user. These online attack capabilities are already included in the online confidentiality definition above, as the attacker who can break the precomputation attack resistance can recover the password and trivially win the online confidentiality experiment by using the recovered password in a decryption protocol on behalf of the honest user.

Definition 4.4 (Precomputation Attack Resistance). A DEEP is called precomputation attack resistant, if for all PPT $\mathcal{A}$ it holds that $\Pr[\text{PAR}_{\mathcal{A},\Gamma}(1^\lambda)] \leq (q_{\text{off}} + 1)/|\mathcal{D}_{pw}| + \text{negl}$ where $\mathcal{D}_{pw}$ is the set of all possible passwords, and q_{off} is the number of queries to $O_{TestPwD}$ after compromising both servers (i.e. $\text{corrDS} = 1$ and $\text{corrKS} = 1$).

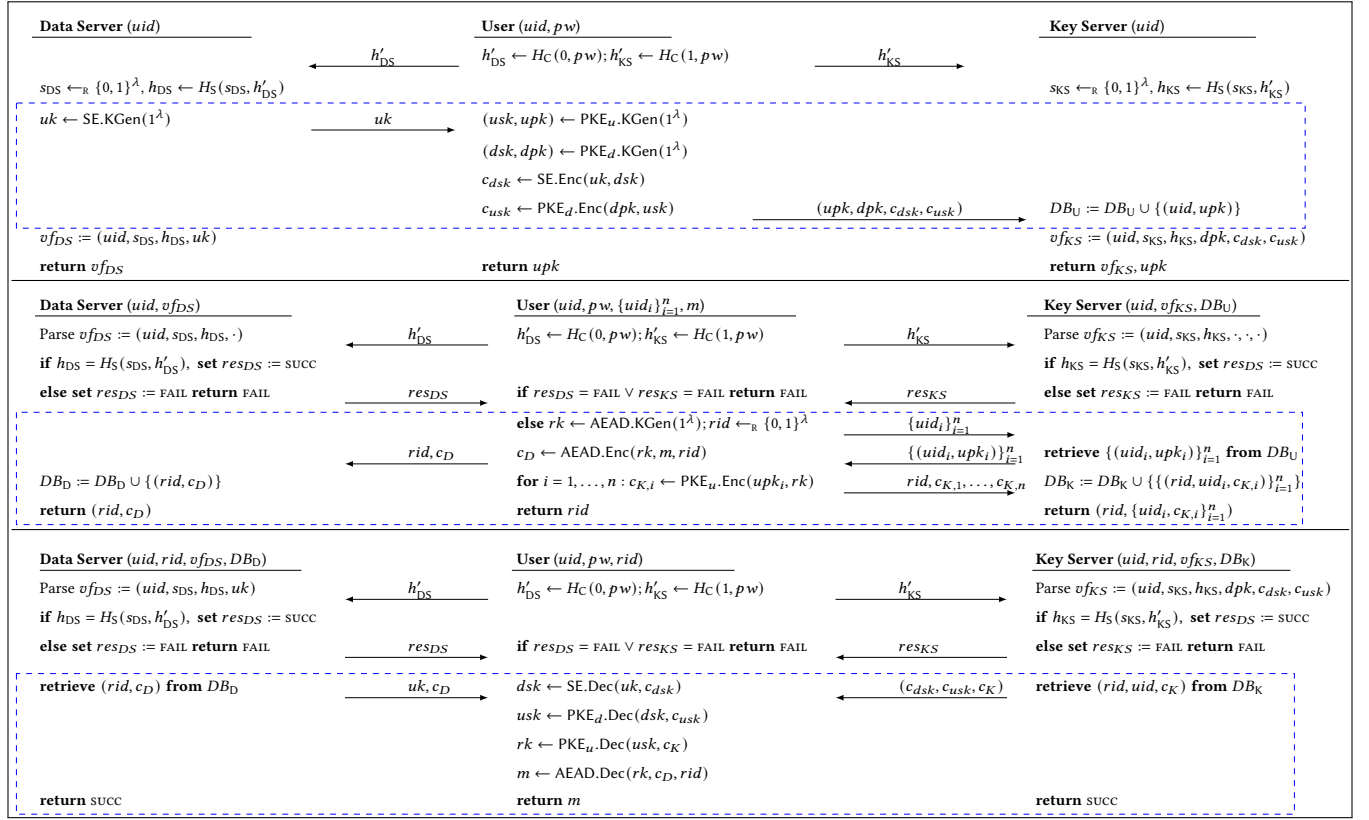


Figure 5: Registration, encryption and decryption algorithms in π_{DEEP} (top to bottom). Parts within the blue dashed boxes are re-used in π_{DEEP}^+ .

5 Security Analysis of Doctolib's π_{DEEP}

With the desired security guarantees for DEEP established, we now analyze the security of Doctolib's protocol. Therefore, we first describe Doctolib's encryption protocol, which we call π_{DEEP} , as an instantiation of DEEP. We then show that the protocol achieves strong confidentiality but does not achieve online confidentiality and conclude with an assessment of the strengths of π_{DEEP} which inspired our improved construction π_{DEEP}^+ in the next section.

5.1 Doctolib's π_{DEEP}

Our interpretation of π_{DEEP} is based on the resources available from Doctolib [19, 33]. We have already outlined the core structure of the protocol in Sec. 3, and present now the concrete algorithms and building blocks. Our focus is on the two main aspects of the protocol: the envelope encryption in the virtual device setting, and the user authentication to get access to the virtual device key.

Building Blocks. π_{DEEP} utilizes the following basic building blocks which were outlined in Sec. 2: A symmetric encryption scheme SE, a symmetric authenticated encryption scheme with associated data AEAD, two public-key encryption schemes PKE_d , PKE_u , and two hash functions H_C , H_S . All communication between the user and the two servers is protected by server-authenticated TLS channels.

Envelope Encryption with Virtual Device. The registration, encryption and decryption algorithms are given in Fig. 5. The protocol uses envelope encryption to encrypt messages, and distributes the virtual device across the two servers as outlined in Sec. 3. In the

envelope encryption process, the user generates a resource identifier rid and uses it as associated data for the AEAD encryption of message m under the resource key rk . Looking ahead, this guarantees a binding between the resource identifier and the ciphertext. This prevents ciphertext injection attacks, where an attacker could re-inject an existing ciphertext from someone else under a different rid and force the recipient to successfully decrypt this ciphertext.

Transparent Double Authentication. The transparent double authentication mechanism to retrieve the virtual device key is implemented with a cascade of password hashes. Upon registration, both servers store a salted hash of a hash of the user's password. That is, the user never reveals her password in clear to the servers, but only sends it in hashed form. The Doctolib whitepaper states that "the password is hashed client-side, using two different hashing algorithms: one for the key server and one for the application server" [19]. The independent hashing is supposed to realize the desired blinding, i.e., guaranteeing that a corrupt server learns no information through the user's authentication that allows it to impersonate the honest user towards the other server.

We abstract the client-side hashing with "two different hash functions" through a single hash function H_C , that is run twice with different prefixes for each server. The server-side hashing utilizes a separate hash function H_S and per-client salts.

5.2 Strong Confidentiality

The envelope encryption mechanism employed in π_{DEEP} allows it to achieve strong confidentiality. Recall that the passive attacker

may only see the ciphertexts created during encryption. These are c_D , which is an AEAD encryption of the message m under a randomly chosen resource key rk , and $c_{K,i}$ which is a public key encryption of rk under each recipient's public key. If the public key encryption scheme PKE_u is IND-CPA secure, the adversary will learn nothing about the resource key from $c_{K,i}$, and if AEAD is IND-EAV secure, the adversary will learn nothing about the underlying message from c_D . CPA-security for PKE_u is sufficient here, as the adversary in this game is purely passive, and for AEAD IND-EAV security suffices since every resource key is only used once. We state the theorem here, the proof is given in App. D.

THEOREM 5.1. *If AEAD is IND-EAV secure and PKE_u is IND-CPA secure, then Doctolib's π_{DEEP} is strongly confidential.*

Note that security guarantees are only needed for two of the four encryption schemes involved, because only AEAD and PKE_u are used in the encryption of a message and the computation of c_D and c_K . In contrast, SE and PKE_d are only used to encrypt and decrypt the user's key material to protect it from the servers.

5.3 Weaknesses of π_{DEEP}

While π_{DEEP} ensures strong confidentiality, it does not achieve online confidentiality or precomputation attack resistance.

5.3.1 No Online Confidentiality. π_{DEEP} does not achieve online confidentiality since the password hashes stored on the servers allow for offline attacks on the password as soon as one of the servers is compromised. Given the typically low entropy of passwords, an attacker has a non-negligible chance of correctly guessing and thus recovering the user's password. Afterward, he can use the password in a decryption query with the other (honest) server to successfully decrypt messages. More precisely, the following adversary $\mathcal{A}$ wins the online confidentiality experiment with probability 1:

$\mathcal{A}^{\text{oracles}}(1^\lambda)$

- Register arbitrary uid_1, uid_2 using $\mathcal{O}_{\text{HReg}}$ oracle
- Pick m_0, m_1 ; send $(uid_1, \{uid_2\}, m_0, m_1, st_0)$ to challenger
- Receive $(rid^*, c_D^*, c_{K,1}^*, \dots, c_{K,n}^*, \perp, st_0)$
- Query $\mathcal{O}_{\text{Corr}}$ (DS) to obtain $\{(uid_1, v_{f_{DS,1}}), (uid_2, v_{f_{DS,2}})\}$
- Parse $v_{f_{DS,2}} := (uid_2, s_{DS}, h_{DS}, uk)$
- Run a brute-force dictionary attack:
- Pick $p_w \in \mathcal{D}_{p_w}$ until $H_S(s_{DS}, H_C(p_w)) = h_{DS}$
- Run $\mathcal{O}_{\text{CDec}}(uid_2, rid^*)$ as in Fig. 5 with p_w and obtain output m
- If $m = m_0$ return 0, else return 1.

This attack involves a brute-force attack, which requires an exhaustive search over the password space $\mathcal{D}_{p_w}$ in the worst case. We assume $\mathcal{D}_{p_w}$ to be polynomially-bounded due to the low entropy of user-chosen passwords, making the attack efficient. Moreover, real-world password cracking algorithms exploit the varying probabilities of passwords which can further optimize the attack [32].

Practical Feasibility of this Attack. We note that common mitigations against password-guessing attacks offer no protection here, as we are concerned with *offline* attacks after a server has been compromised. Salting (which is done as part of the protocol) prevents pre-computation attacks before the server gets compromised, or using a single dictionary for all users, but does not protect against targeted offline attacks after the server compromise. CAPTCHAs are also ineffective, as they only slow down online guessing attacks, but offer no defense against offline attacks.

Enforcing two-factor authentication (2FA) could prevent this attack, as the attacker then additionally needs to compromise a second factor to impersonate a user. While the Doctolib app supports 2FA, it is unclear whether this feature is mandatory for all users and all logins. We therefore did not model 2FA and focused on security in the password-only setting.

Using slow hash functions can also harden the attack. Tanker recommends the use of Blake2b for the client-side hash function H_C , and Argon2 for the servers' hash function H_S [34]. Argon2 [10] is a memory-hard hash function, which significantly raises the computational effort required for an offline attack. Nevertheless, a variation of our attack bypasses Argon2's memory-hardness: If an attacker compromises a server and observes the login of an honest client, he learns the client-side hash $H_C(\cdot, pw)$ and can run a dictionary attack against this hash value. This attack is efficient as Blake2b is a fast, non-memory hard hash function, and this hash is even unsalted – making it vulnerable to pre-computation attacks.

Two-for-one Password Guesses. The previous attack relies on the hashes being exposed to the adversary after compromising one of the servers. However, even in case the servers are never compromised, π_{DEEP} does not achieve online confidentiality because an attacker can validate two password guesses within each encryption or decryption query. This is because the user authenticates *independently* with each server. More precisely, the following PPT attacker breaks the online confidentiality:

$\mathcal{A}^{\text{oracles}}(1^\lambda)$

- Register arbitrary uid_1, uid_2 using $\mathcal{O}_{\text{HReg}}$ oracle
- Pick m_0, m_1 ; send $(uid_1, \{uid_2\}, m_0, m_1, st_0)$ to challenger
- Receive $(rid, c_D^*, c_{K,1}^*, \dots, c_{K,n}^*, \perp, st_0)$; set $p_w^* := \perp$
- For $i = 1, \dots, t$:
 - Pick $p_{w_{i,1}} \leftarrow \mathcal{D}_{p_w}, p_{w_{i,2}} \leftarrow \mathcal{D}_{p_w}$;
 - Run $\mathcal{O}_{\text{CDec}}(uid_2, rid)$ as in Fig. 5, but use $p_{w_{i,1}}$ with DS and $p_{w_{i,2}}$ with KS
 - If DS returns succ, exit and set $p_w^* := p_{w_{i,1}}$
 - If KS returns succ, exit and set $p_w^* := p_{w_{i,2}}$
- Run $\mathcal{O}_{\text{CDec}}(uid_2, rid^*)$ as in Fig. 5 with p_w^* to obtain m
- If $m = m_0$ return 0, else return 1.

This attacker clearly runs in polynomial time if t is polynomial in 1^λ . In each of the first t decryption queries, one of the two passwords could be correct, so the success probability is $1/2 + 2t/|\mathcal{D}_{p_w}|$, while the attacker only makes $t + 1$ online queries to the $\mathcal{O}_{\text{CDec}}$ oracle. For any $t > 1$, π_{DEEP} is thus not online confidential, even if we consider that the servers are never compromised.

5.3.2 No Precomputation Attack Resistance. A similar attack strategy as in Sec. 5.3.1 also breaks the precomputation attack resistance of π_{DEEP} . Recall that the attacker in the precomputation attack resistance experiment may corrupt servers adaptively but should not be able to make offline guesses against the password before compromising both servers. However, obtaining the password hash in π_{DEEP} after compromising one server allows the attacker to perform offline password guesses against a user's password. Therefore, π_{DEEP} is not resistant to precomputation attacks.

5.4 Design Strengths

While we have identified some weaknesses of π_{DEEP} , its encryption approach remains fundamentally sound. As indicated by the

strong confidentiality property, the envelope encryption – combining AEAD encryption for the actual data using rid as associated data, and public key encryption for the resource key – effectively protects the data from passive attackers. The main issue we found lies in the authentication mechanism introduced by the virtual device with its password-hash-based authentication mechanism which allows for offline attacks after compromising just one server.

To address this issue, in the next section, we propose an improved scheme π_{DEEP}^+ which retains the same encryption structure as π_{DEEP} , but introduces an enhanced authentication mechanism using a two-server Distributed Oblivious Pseudorandom Function (dOPRF). Our goal is to ensure that both servers must be compromised before offline attacks can be executed, and to ensure that only one password may be tested per interaction. This showcases that the encryption structure of Doctolib's π_{DEEP} is sound, and that it can be upgraded with minimal computational overhead to achieve the best possible security guarantees.

6 Protocol π_{DEEP}^+

In this section, we present an improved protocol – π_{DEEP}^+ – which achieves the security goals of strong and online confidentiality and is resistant to precomputation attacks. We first describe its building blocks, then present π_{DEEP}^+ , and conclude with the security analysis.

6.1 Building Blocks

Encryption. Our protocol π_{DEEP}^+ uses the same encryption scheme building blocks as π_{DEEP} : a symmetric AEAD scheme AEAD, a symmetric encryption scheme SE, and two public key encryption schemes PKE_d , PKE_u . For online confidentiality, we require the additional security properties of 1-AEAD security and robustness from AEAD (see Sec. 2 and App. A for the definition). Our definition of robustness (sometimes referred to as key-committing encryption [1, 14]) is implied by the full robustness of Farshim et al. [22] where the adversary can choose the ciphertext that must decrypt properly under two distinct keys. While XChaCha20Poly1305 (which is used by Tanker [34]) does not have this key-committing property, it can be transformed into a key-committing encryption scheme with minimal computational overhead [1, 9, 14].

Distributed 2HashDH. π_{DEEP}^+ uses a distributed oblivious pseudorandom function (dOPRF) to transform a password into a high-entropy value. Concretely, we use the 2-server 2HashDH protocol [27] which works in a group $\mathbb{G}$ of prime order q and uses hash functions H_1 and H_2 . Each server holds an additive key share $k_i \in \mathbb{Z}_q$. A user then interacts with both servers to blindly compute $y := H_2(x, H_1(x)^{k_1+k_2})$ for input x . To obtain this value, the user first sends the blinded input $\tilde{x} \leftarrow H_1(x)^b$ for a random $b \in \mathbb{Z}_q$ to each server. Each server computes $\tilde{y}_i \leftarrow \tilde{x}^{k_i}$ and sends $\tilde{y}_i$ to the user who unblinds both values using b and gets the final output as $y \leftarrow H_2(x, (\tilde{y}_1 \cdot \tilde{y}_2)^{1/b})$. 2HashDH is *pseudorandom*, i.e. the output y is indistinguishable from that of a random function for anyone who does not possess both servers' key shares. It is also *oblivious*, meaning the servers learn nothing about the user's input x . Further, the user can evaluate x only when interacting with both servers and no single server can evaluate the function themselves.

2HashDH realizes these properties in the RO model under the Gap OMDH assumption. Briefly, this assumption states that it is

hard for an adversary, given access to a k -th power oracle Q times, to output the k -th power of $Q+1$ different group elements, all while having access to a DDH oracle. We give a formal definition of the assumption in Appendix B.

Signature with Explicit Randomness. We further utilize a signature scheme SIG consisting of a setup algorithm $pp \leftarrow \text{Setup}(1^\lambda)$, key generation $(sk, pk) := \text{KGen}(pp; r)$ for randomness r , a sign algorithm $\sigma \leftarrow \text{Sign}(sk, m)$ and verify algorithm $b \leftarrow \text{Vf}(pk, m, \sigma)$. Note that KGen is a deterministic function when a randomness $r \in \mathcal{R}_\lambda$ (where $\mathcal{R}_\lambda$ denotes the randomness space) is given as input. We require the scheme to be existentially unforgeable under chosen message attacks (EUF-CMA): It must be infeasible for an adversary given pk from $(sk, pk) := \text{KGen}(pp; r)$ for random $r \leftarrow_{\mathcal{R}_\lambda}$ and access to a sign oracle to create a valid signature on a fresh message.

6.2 Protocol

The core idea of π_{DEEP}^+ is that the user, with the (oblivious) assistance of the servers, deterministically transforms a low-entropy password pw into a high-entropy signature key-pair and then proves possession of the secret key to authenticate. Overall, our approach follows prior works [4, 7, 16] which have successfully combined a (distributed) OPRF with an unforgeable signature scheme (or MAC) to achieve user authentication resistant to offline attacks. We outline the protocols for registration, encryption and decryption which are given in Fig. 7 in App. B.

Registration. The user generates a dOPRF key share for each server (k_{DS} for DS and k_{KS} for KS) and derives a signature key pair (sk_s, pk_s) using the 2HashDH evaluation $H_2(pw, H_1(pw)^{k_{\text{DS}}+k_{\text{KS}}})$ as randomness. The user sends pk_s to both servers, and each key share to the respective server. The registration proceeds with the same key creation process as in π_{DEEP} , i.e. the keys uk , usk , upk , dsk , and dpk are generated and managed as described in π_{DEEP} (Fig. 5). Importantly, the user needs to safely delete the dOPRF key shares after registration to avoid offline attacks.

Authentication During Encryption and Decryption. In π_{DEEP}^+ , we modify the authentication at the beginning of the encryption and decryption protocol while keeping the rest of π_{DEEP} 's algorithms unchanged. To authenticate, the user runs the dOPRF protocol with both servers to evaluate 2HashDH on her password and deterministically derive the signature key pair, which will match the one created during registration if the password is correct. The user then proves knowledge of this secret key through a challenge-response process: each server sends a challenge message, which the user signs and returns. The servers verify the signatures using the signature public key stored during registration.

Security Idea. An attacker who compromises one server (and learns (pk, k_{DS}) or (pk, k_{KS})) cannot use this information to authenticate to the other server due to the unforgeability of the signature scheme and the pseudorandomness of the dOPRF. Brute-forcing the password requires evaluating 2HashDH with both key shares, forcing the attacker to either compromise both servers or interact with the second server – making this an online attack. Moreover, password tests require engaging with both servers using the *same* password, thus avoiding the 2-for-1 password guesses from π_{DEEP} .

Note that while our approach requires the servers to store keys, these are user-specific and must be protected in the same manner

as user-specific hashes and salts in the hash-based authentication from π_{DEEP} . Therefore, this does not introduce any additional trust assumptions for the servers.

Simplifying the Encryption? In π_{DEEP}^+ , we adopt the same encryption structure as in π_{DEEP} where users retrieve their user secret keys through a cascade of decryption steps with each step unpacking a new key. One of the peaks of this design choice is that it allows multiple devices to use the same user key usk , as usk is encrypted separately for each device. However, in the virtual device setting, we assume that users only have a single device – the virtual device, and this step of encrypting the usk towards every device becomes obsolete. In fact, we can simplify the encryption process – DS stores uk , and KS stores the encrypted usk , encrypted directly under uk , thus removing the need for device keys – and maintain the same level of security. Since the confidentiality of usk remains protected by the symmetric encryption scheme SE, this modification would yield a secure DEEP in the single-device setting. We opted to retain π_{DEEP} 's structure to maintain compatibility with physical devices.

6.3 Security Analysis

We show that π_{DEEP}^+ achieves all our desired security properties. We begin with strong confidentiality which essentially carries over from the proof of π_{DEEP} before showing online confidentiality and precomputation attack resistance.

6.3.1 Strong Confidentiality. Since the attacker remains passive in this case, and does not learn any intermediate values of any of the protocols in the security experiment, the attacker is actually oblivious to which authentication procedure is used in the protocol. Therefore, the strong confidentiality follows straightforward from the strong confidentiality of π_{DEEP} (Thm. 5.1).

THEOREM 6.1. *If AEAD is IND-EAV secure and PKE_u is IND-CPA secure, then π_{DEEP}^+ is strong confidential.*

6.3.2 Online Confidentiality. We now consider security in the presence of active attackers. To see why π_{DEEP}^+ is online confidential, we explain the impossibility of different attack venues:

Impersonating a recipient: The attacker can impersonate a user, if he guesses the password of the user correctly in a query to O_{CDec} . However, due to the security of the distributed OPRF, the attacker can only derive one valid signing key per interaction with the servers, limiting the attacker to one password guess.

Deriving the message from the challenge ciphertexts: π_{DEEP}^+ uses the same encryption structure as π_{DEEP} , thus the ciphertexts c_D, c_K still protect the message from passive eavesdroppers.

Injecting the challenge ciphertexts: An attacker could receive a ciphertext and inject parts of it as a ciphertext on behalf of a different user towards the same recipient uid under a different rid . Then, he could force the uid to decrypt and obtain the message. However, this decryption will fail with overwhelming probability as the ciphertext cannot decrypt properly for two distinct $rids$ (due to one-time ciphertext integrity of AEAD) or two distinct keys (due to robustness of AEAD).

Compromising one of the servers: If a server is compromised, the stored data does not reveal information about the underlying keys due to the IND-CPA security of PKE_d and the IND-EAV security of SE. The unforgeability of the signature scheme means

that compromising one server does not allow to authenticate as the user towards the other server, and the security of the dOPRF makes it so that compromising one server still requires online interacting with the other server to verify a password guess.

We now provide the formal theorem, the proof is given in App. D.

THEOREM 6.2. *If AEAD is IND-EAV secure, robust and has one-time ciphertext integrity, if PKE_u is IND-CCA secure, SE is IND-EAV secure, PKE_d is IND-CPA secure, if SIG is an unforgeable signature scheme, if H_1 and H_2 are modeled as random oracles and if the (N, Q) -OMDH-assumption holds in $\mathbb{G}$, then π_{DEEP}^+ is online confidential.*

6.3.3 Precomputation Attack Resistance. For precomputation attack resistance, we rely on the outer hash function H_2 of 2HashDH to behave like a random oracle. Any attacker that receives an dOPRF output, cannot immediately derive its input since it is the output of the random oracle H_2 , and each password needs to be tested. The proof for the following theorem is given in App. D.

THEOREM 6.3. *The π_{DEEP}^+ protocol is precomputation attack resistant, if H_2 is modeled as a random oracle.*

6.4 Efficiency

We compare π_{DEEP}^+ and Doctolib's π_{DEEP} in terms of efficiency. As both protocols use the same encryption structure, we only need to consider the authentication part. The 2HashDH approach of π_{DEEP}^+ adds one communication round, and replaces 2 hash computations on the user side with 2 group exponentiations and two signature creations. Each server computes one group exponentiation and one signature verification in π_{DEEP}^+ instead of 1 hash value in π_{DEEP} . Overall, π_{DEEP}^+ adds one round of communication, 4 group exponentiations, and 2 signature creations and verifications to each encryption and decryption request. Concretely, using the available MIRACL library benchmarks [31] with ECDSA on a 256-bit curve, the combined computational overhead is less than 15ms across all three parties. Our solution therefore adds only minimal overhead compared with π_{DEEP} yet provides significantly stronger security.

7 Conclusion

In this work, we conducted the first formal security analysis of Doctolib's distributed envelope encryption protocol. To do this, we introduced the concept of password-based distributed envelope encryption and defined game-based security notions addressing optimal confidentiality guarantees against both passive and malicious attackers, while maintaining Doctolib's usability requirements. Our analysis shows that π_{DEEP} ensures strong confidentiality against passive attackers. However, when attackers are active, the protocol does not provide the optimal confidentiality guarantees that are possible in such a system. We finally showed that an improved version of the protocol, π_{DEEP}^+ , meets all security objectives.

Our focus was on evaluating the overall cryptographic design, and our analysis assumes all parties use secure instantiations of the abstracted building blocks. A valuable next step would be to evaluate Doctolib's real-world implementation, similar to the studies conducted on MEGA [3, 5] and Nextcloud [2]. Moreover, investigating the integrity of data stored by the servers, and particularly the security guarantees of Doctolib's trustchain, would be a great complement to our work which focused on confidentiality.

Additionally, Doctolib offers advanced features not covered in this work, such as group sharing, sharing with unregistered users, multi-device and physical device access. Analyzing these features could provide valuable insights into DEEP's framework.

References

- [1] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. 2022. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*. 3291–3308.
- [2] Martin Albrecht, Matilda Backendal, Daniele Coppola, and Kenneth G Paterson. 2024. Share with Care: Breaking E2EE in Nextcloud. In *Euro S&P 2024*.
- [3] Martin R Albrecht, Miro Haller, Lenka Mareková, and Kenneth G Paterson. 2023. Caveat implementor! Key recovery attacks on MEGA. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 190–218.
- [4] Matilda Backendal, Hannah Davis, Felix Günther, Miro Haller, and Kenneth G Paterson. 2024. A formal treatment of end-to-end encrypted cloud storage. In *Annual International Cryptology Conference*. Springer, 40–74.
- [5] Matilda Backendal, Miro Haller, and Kenneth G Paterson. 2023. MEGA: malleable encryption goes awry. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 146–163.
- [6] Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. 2011. Password-protected secret sharing. In *Proceedings of the 18th ACM conference on Computer and Communications Security*. 433–444.
- [7] Carsten Baum, Tore Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai. 2020. PESTO: proactively secure distributed single sign-on, or how to trust a hacked server. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 587–606.
- [8] Rebecca Beerheide, André Haserück, and Eva Richter-Kuhlmann. 2021. Coronapandemie: Impfstart mit Hindernisse. <https://www.aerzteblatt.de/archiv/217259/Coronapandemie-Impfstart-mit-Hindernissen>. Last accessed on 30-11-2023.
- [9] Mihir Bellare and Viet Tung Hoang. 2024. Succinctly-committing authenticated encryption. In *Annual International Cryptology Conference*. Springer, 305–339.
- [10] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. 2016. Argon2: new generation of memory-hard functions for password hashing and other applications. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 292–302.
- [11] Dan Boneh and Victor Shoup. 2023. A Graduate Course in Applied Cryptography. (2023).
- [12] Xavier Boyen. 2009. Hidden credential retrieval from a reusable password. In *Proceedings of the 4th International Symposium on Information, Computer, and Communications Security*. 228–238.
- [13] Jan Camenisch, Anja Lehmann, and Gregory Neven. 2015. Optimal distributed password verification. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 182–194.
- [14] John Chan and Phillip Rogaway. 2022. On committing authenticated-encryption. In *European Symposium on Research in Computer Security*. Springer, 275–294.
- [15] Long Chen, Ya-Nan Li, Qiang Tang, and Moti Yung. 2022. {End-to-Same-End} Encryption: Modularly Augmenting an App with an Efficient, Portable, and Blind Cloud Storage. In *31st USENIX Security Symposium (USENIX Security 22)*. 2353–2370.
- [16] Poulami Das, Julia Hesse, and Anja Lehmann. 2022. DPASE: distributed password-authenticated symmetric-key encryption, or how to get many keys from one password. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. 682–696.
- [17] Dennis Dayanikli and Anja Lehmann. 2023. Password-based credentials with security against server compromise. In *European Symposium on Research in Computer Security*. Springer, 147–167.
- [18] Romain Dillet. 2022. Healthcare tech platform Doctolib acquires encryption startup Tanker. <https://techcrunch.com/2022/01/11/healthcare-tech-platform-doctolib-acquires-encryption-startup-tanker/>. Last accessed on 29-11-2023.
- [19] Doctolib. 2022. A New Standard for Online Medical Confidentiality. https://media.doctolib.com/image/upload/mkg/file/Doctolib_whitepaper_-_A_new_standard_for_online_medical_confidentiality.pdf Whitepaper.
- [20] Doctolib. 2022. Press Release - Doctolib acquires Tanker and its end-to-end encryption technology to set a new standard for online medical confidentiality. https://media.doctolib.com/image/upload/mkg/file/EN_220121_Doctolib_acquires_Tanker. Last accessed on 20-09-2024.
- [21] Doctolib. 2024. About Doctolib. <https://about.doctolib.com/>. Last accessed on 06-27-2024.
- [22] Pooya Farshim, Claudio Orlandi, and Răzvan Roşie. 2017. Security of symmetric primitives under incorrect usage of keys. *IACR Transactions on Symmetric Cryptology*, 2017(1) (2017), 449–473.
- [23] The Local France. 2021. French government partners with Doctolib for Covid-19 vaccine appointments. [https://www.thelocal.fr/20210112/french-](https://www.thelocal.fr/20210112/french-government-partners-with-doctolib-to-allow-people-to-book-covid-19-vaccine-appointments-online/)

government-partners-with-doctolib-to-allow-people-to-book-covid-19-vaccine-appointments-online/. Last accessed on 30-11-2023.

- [24] Shioichi Hirose and Kazuhiko Minematsu. 2023. A Formal Treatment of Envelope Encryption. *Cryptology ePrint Archive* (2023).
- [25] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. 2014. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In *Advances in Cryptology-ASIACRYPT 2014: 20th International Conference on the Theory and Application of Cryptology and Information Security*, Kaoshiung, Taiwan, ROC, December 7-11, 2014, *Proceedings, Part II* 20. Springer, 233–253.
- [26] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. 2016. Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online). In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 276–291.
- [27] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. 2017. TOPPSS: cost-minimal password-protected secret sharing based on threshold OPRF. In *Applied Cryptography and Network Security: 15th International Conference, ACNS 2017, Kanazawa, Japan, July 10-12, 2017, Proceedings* 15. Springer, 39–58.
- [28] Stanislaw Jarecki, Hugo Krawczyk, and Jason Resch. 2019. Updatable oblivious key management for storage systems. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 379–393.
- [29] Mallory Knodel, Sofia Celi, Olaf Kolkman, and Gurshabad Grover. 2024. Definition of End-to-end Encryption. *Cryptology ePrint Archive*, Paper 2024/2085. <https://eprint.iacr.org/2024/2085>
- [30] Romain Mekarni. 2022. An introduction to encryption: how to protect data in healthcare. <https://medium.com/doctolib/an-introduction-to-encryption-how-to-protect-data-in-healthcare-4d3677add6ec>. Last accessed on 20-09-2024.
- [31] MIRACL. 2024. MIRACL Crypto Library for High-Performance. <https://github.com/miracrl/MIRACL/blob/master/docs/miracrl-explained/benchmarks.md>. Last accessed on 17-09-2024.
- [32] Ruixin Shi, Yongbin Zhou, Yong Li, and Weili Han. 2021. Understanding Offline Password-Cracking Methods: A Large-Scale Empirical Study. *Security and Communication Networks* 2021, 1 (2021), 5563884.
- [33] Tanker. 2023. Technical specification of the Tanker SDK and protocol for end-to-end encryption. <https://github.com/TankerHQ/spec>. Last accessed on 28-11-2023.
- [34] Tanker. 2024. Technical specification of the Tanker SDK and protocol for end-to-end encryption Overview. <https://github.com/TankerHQ/spec/blob/master/overview.md>. Last accessed on 20-09-2024.

A Security Properties of Building Blocks

Here, we provide formal definitions for the security properties used in our paper.

A.1 AEAD Security Properties

IND-EAV security for AEAD is a straightforward extension of IND-EAV security for regular symmetric encryption, which additionally takes in associated data ad .

Definition A.1 (IND-EAV). An AEAD scheme $\pi = (\text{KGen}, \text{Enc}, \text{Dec})$ is IND-EAV secure if for every PPT adversary $\mathcal{A}$ it holds that $\Pr[\text{IND}_{\mathcal{A},\pi}^{\text{EAV}}(\lambda) = 1] \leq 1/2 + \text{negl}$ where the experiment $\text{IND}_{\mathcal{A},\pi}^{\text{EAV}}(\lambda)$ is defined in Fig. 6.

One-time ciphertext integrity means that the adversary, given a ciphertext under an unknown key k for (m, ad) cannot provide a different ciphertext-associated data pair (c^*, ad^*) that decrypts properly under k . If the encryption scheme provides both IND-EAV security and one-time ciphertext integrity, we call it 1-AEAD secure. This is the AEAD equivalent of 1-AE security defined in [11].

Definition A.2 (1-CI). An AEAD scheme $\pi = (\text{KGen}, \text{Enc}, \text{Dec})$ has one-time ciphertext integrity (1-CI) if for every PPT adversary $\mathcal{A}$ it holds that $\Pr[1 - \text{CI}_{\mathcal{A},\pi}(\lambda) = 1] \leq \text{negl}$ where the experiment $1 - \text{CI}_{\mathcal{A},\pi}(\lambda)$ is defined in Fig. 6.

Definition A.3 (1-AEAD). An encryption scheme $\pi = (\text{KGen}, \text{Enc}, \text{Dec})$ is 1-AEAD secure if it has one time ciphertext integrity and IND-EAV security.

Experiment $\text{IND}_{\mathcal{A},\Pi}^{\text{EAV}}(\lambda)$	Experiment $\text{ROB}_{\mathcal{A},\Pi}(\lambda)$	Experiment $1 - \text{CI}_{\mathcal{A},\Pi}(\lambda)$	Experiment $\text{IND}_{\mathcal{A},\Pi}^x(\lambda)$ for $x \in \{\text{CPA}, \text{CCA}\}$
$k \leftarrow_{\mathcal{R}} \text{KGen}(1^\lambda)$ $(m_0, m_1, ad) \leftarrow \mathcal{A}(1^\lambda)$ $b \leftarrow_{\mathcal{R}} \{0, 1\}; c^* \leftarrow \text{Enc}(k, m_b, ad)$ $b' \leftarrow \mathcal{A}(c^*)$ return 1 if $(b = b' \wedge m_0 = m_1)$	$k \leftarrow_{\mathcal{R}} \text{KGen}(1^\lambda)$ $(m, ad) \leftarrow \mathcal{A}(1^\lambda)$ $c \leftarrow \text{Enc}(k, m, ad)$ $(k^*, ad^*) \leftarrow \mathcal{A}(c)$ return 1 if $k \neq k^* \wedge \text{Dec}(k^*, c, ad^*) \neq \perp$	$k \leftarrow_{\mathcal{R}} \text{KGen}(1^\lambda)$ $(m, ad) \leftarrow \mathcal{A}(1^\lambda)$ $c \leftarrow \text{Enc}(k, m, ad)$ $(c^*, ad^*) \leftarrow \mathcal{A}(c)$ return 1 if $(c^*, ad^*) \neq (c, ad) \wedge \text{Dec}(k^*, c^*, ad^*) \neq \perp$	$(sk, pk) \leftarrow_{\mathcal{R}} \text{KGen}(1^\lambda)$ $(m_0, m_1) \leftarrow \mathcal{A}^{\text{Dec}(sk, \cdot)}(pk)$ $b \leftarrow_{\mathcal{R}} \{0, 1\}; c^* \leftarrow \text{Enc}(pk, m_b)$ $b' \leftarrow \mathcal{A}^{\text{Dec}(sk, \cdot)}(c^*)$ return 1 if $(b = b' \wedge c^* \text{ was never queried to } \text{Dec}(sk, \cdot))$

Figure 6: Security experiments for IND-EAV security, robustness, and one-time ciphertext integrity (1-CI) for an AEAD scheme, as well as IND-CPA and IND-CCA (including **highlighted text) security for a PKE scheme.**

Another property that we define is robustness. Robustness ensures that it is difficult for an adversary, given a ciphertext c of (m, ad) encrypted under an unknown key k , to find a key $k^* \neq k$ and ad^* such that c decrypts properly under k^* and ad^* .

Definition A.4 (Robustness). An AEAD scheme $\pi = (\text{KGen}, \text{Enc}, \text{Dec})$ is robust if for every PPT adversary $\mathcal{A}$ it holds that

$$\Pr[\text{ROB}_{\mathcal{A},\Pi}(\lambda) = 1] \leq \text{negl}$$

where the experiment $\text{ROB}_{\mathcal{A},\Pi}(\lambda)$ is defined in Fig. 6.

Our definition is a weaker variant of full robustness from Farshim et al. [22] where the adversary can select the ciphertext that must decrypt properly under two distinct keys.

A.2 PKE Security Properties

We recall the definitions for IND-CPA and IND-CCA security.

A PKE scheme is IND-CPA secure, if an attacker, when given a public key pk and a challenge ciphertext c of a message, should not be able to derive any information about the underlying plaintext m . In this scenario, the attacker can obtain multiple encryptions of arbitrary messages as he knows the public key pk .

Definition A.5 (IND-CPA Security). A PKE scheme $\pi = (\text{KGen}, \text{Enc}, \text{Dec})$ is IND-CPA secure if for every PPT adversary $\mathcal{A}$ it holds that $\Pr[\text{IND}_{\mathcal{A},\Pi}^{\text{CPA}}(\lambda) = 1] \leq 1/2 + \text{negl}$ where the experiment $\text{IND}_{\mathcal{A},\Pi}^{\text{CPA}}(\lambda)$ is defined in Fig. 6.

In the IND-CCA security experiment, the attacker additionally has access to a decryption oracle which decrypts arbitrary ciphertexts of the attacker's choice, under the condition that they are not the challenge ciphertext. Still, he should be unable to derive any information about the underlying message m .

Definition A.6 (IND-CCA Security). A PKE scheme $\pi = (\text{KGen}, \text{Enc}, \text{Dec})$ is IND-CCA secure if for every PPT adversary $\mathcal{A}$ it holds that $\Pr[\text{IND}_{\mathcal{A},\Pi}^{\text{CCA}}(\lambda) = 1] \leq 1/2 + \text{negl}$ where the experiment $\text{IND}_{\mathcal{A},\Pi}^{\text{CCA}}(\lambda)$ is defined in Fig. 6.

B π_{DEEP}^+ & Gap OMDH Assumption

In this section, we provide the concrete algorithms for our improved π_{DEEP}^+ and elaborate on the Gap OMDH assumption. The algorithms for registration and authentication during encryption and decryption are given in Fig. 7.

The security of the 2HashDH distributed OPRF used in our paper relies on the Gap One More Diffie Hellman (Gap OMDH) assumption, which we briefly recall here.

Definition B.1 ((N, Q)-Gap OMDH assumption [26]). Let $\mathbb{G}$ be a group of prime order q , and let g be a generator of $\mathbb{G}$. The (N, Q) one more Diffie Hellman (OMDH) states that for any PPT adversary $\mathcal{A}$, the probability

$$\Pr[(g_{j_i}, g_{j_i}^k)_{i=1,\dots,Q+1} \leftarrow \mathcal{A}^{(\cdot)^k, \text{DDH}(\cdot, \cdot)}(g, g^k, g_1, \dots, g_N)]$$

is negligible, where $k \leftarrow_{\mathcal{R}} \mathbb{Z}_q$, $g_1, \dots, g_N \leftarrow_{\mathcal{R}} \mathbb{G}$, $(\cdot)^k$ is a k -th power oracle, DDH is a DDH oracle, Q is the number of $\mathcal{A}$'s queries to $(\cdot)^k$ and all j_i are distinct elements in $\{1, \dots, N\}$.

In other words, consider an adversary $\mathcal{A}$ with access to a “ k -th power” oracle and a DDH oracle with the constraint that it can only make Q queries to the $(\cdot)^k$ oracle. If $\mathcal{A}$ is given N random elements from $\mathbb{G}$ and queries the $(\cdot)^k$ oracle Q times, $\mathcal{A}$ can compute the k -th power for any Q out of the N elements. The Gap OMDH assumption states that the probability of $\mathcal{A}$ being able to compute the k -th power for any $Q+1$ of the N elements (i.e., for one additional element) is negligible.

C Definition of Correctness

We now define the correctness of a DEEP. Correctness models the expected functional behavior in executions of honestly behaving clients and servers. Given that there are multiple clients in the system which can execute the encryption and decryption of data at any point, and in arbitrary order, we do this via the game Corr given in Fig. 8. The key intuition of this game is that if a user uid successfully encrypts a file m for some recipient uid' with resource identifier rid , then, uid' should, upon successfully decrypting the ciphertext associated with rid , recover the original message m . Formally, this is captured by letting the adversary win the game if the adversary can give an rid and two users (uid_0, uid_1) such that uid_0 encrypted a message m for recipient uid_1 but the decryption of uid_1 yields a different message m' .

We model the correctness game using three oracles $\mathcal{O}_{\text{Reg}}$, $\mathcal{O}_{\text{Enc}}$, $\mathcal{O}_{\text{Dec}}$ which allows an adversary to trigger honest executions of the algorithms on adversarially chosen inputs. Notably, we let the adversary choose the passwords of all parties, modeling the users potentially mistyping their passwords. Whenever a message is successfully encrypted or decrypted, it gets logged in $\mathcal{Q}_{\text{Enc}}$ and $\mathcal{Q}_{\text{Dec}}$, respectively, along with the associated $uids$ and rid . The

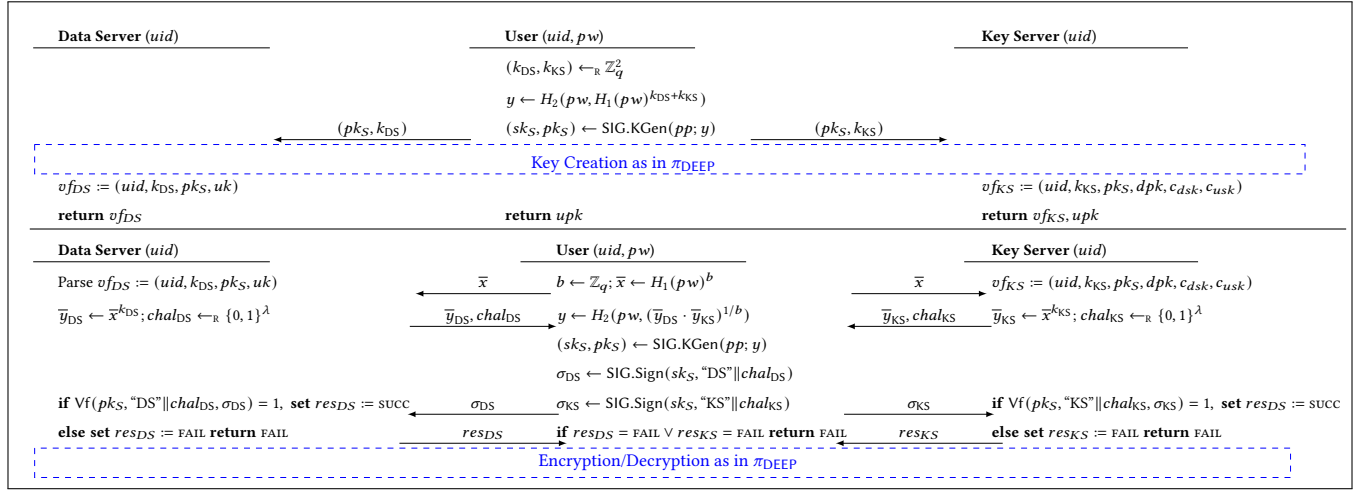


Figure 7: Registration (top) and user authentication within encryption and decryption (bottom) in π_{DEEP}^+ . The key creation part during registration, and the encryption and decryption follow as in π_{DEEP} and are highlighted in Fig. 5.

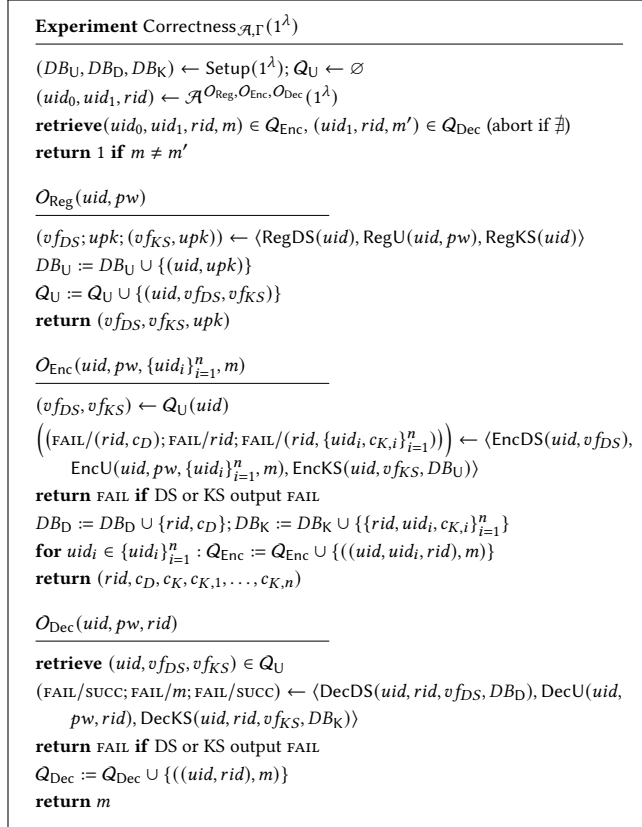


Figure 8: Security Experiment for Correctness.

adversary wins if he can find an rid and two users uid_0, uid_1 such that the message encrypted from uid_0 to uid_1 does not match the message decrypted by uid for rid .

Definition C.1 (Correctness). A Distributed Envelope Encryption (DEEP) $\Gamma = (\text{Setup}, \text{Reg}, \text{Enc}, \text{Dec})$ is *correct* if for all probabilistic polynomial-time adversaries $\mathcal{A}$ it holds that $\Pr[\text{Correctness}_{\mathcal{A}, \Gamma}(1^\lambda) = 1] \leq \text{negl}$.

D Proofs

D.1 Proof of Theorem 5.1

Proof Idea. We recall that in the setting of strong confidentiality, all parties are honest, while the adversary remains entirely passive, restricted to initiating honest executions of the protocol between the parties and learn the outputs of the encryption and decryption algorithms. In the security experiment, the adversary can query the encryption oracle and receive ciphertexts of the form $c_D = \text{AEAD}.\text{Enc}(rk, m, rid)$ and $c_{K,i} = \text{PKE}_u.\text{Enc}(upk_i, rk)$. While the adversary has access to a decryption oracle, he can only use it to decrypt honestly generated ciphertexts for which the adversary knows the message, thus giving the adversary no additional information. At some point the adversary receives the challenge ciphertexts $c_D^* = \text{AEAD}_r.\text{Enc}(rk, m_b, rid^*)$, $\{c_{K,i}^* = \text{PKE}_u.\text{Enc}(upk_i, rk)\}_{i=1}^n$ and must decide whether $b = 0$ or $b = 1$. Roughly, the adversary has two ways of learning information about m_b : (1) directly from c_D^* which is an encryption of m_b under a randomly chosen resource key rk , or (2) by learning information about rk from the ciphertexts $c_{K,i}^*$ which contain rk encrypted under each recipient's public key upk_i . Attack venue (2) is not possible, if PKE_u is IND-CPA-secure, and attack venue (1) is not possible if AEAD is IND-EAV secure. Note that the resource keys are never reused, and the adversary thus never sees two ciphertexts encrypted under the same rk , which is why IND-EAV security is enough for AEAD.

Formal Proof. Let $\mathcal{A}$ be an efficient adversary in the strong confidentiality experiment of Doctolib's DEEP Γ . Recall that $\mathcal{A}$ has access to the oracles for honest protocol execution O_{HReg} , O_{HEnc} , O_{HDec} , and has to determine whether the challenge ciphertexts $c_D^*, c_{K,1}^*, \dots, c_{K,n}^*$ from some honest uid to honest recipients uid^* decrypts to message m_0 or m_1 . We prove security with a hybrid

argument. Let G_0 be the strong confidentiality experiment with the real Doctolib protocol. We subsequently change the experiment until we end up with Game G_2 , where there is no information about m_b and the adversary can therefore only guess the bit b correctly with probability $1/2$. We show the game hops to be indistinguishable if AEAD is IND-EAV secure, and PKE_u is IND-CPA secure. Let S_i be the event that $\mathcal{A}$ wins the confidentiality experiment in game G_i , and let Adv_i be the advantage of $\mathcal{A}$, i.e. $\text{Adv}_i = |S_i - 1/2|$

Game G_0 : Game G_0 corresponds to the real strong unforgeability experiment between $\mathcal{A}$ and the challenger outlined in Fig. 3.

Game G_1 : *Encrypt random key r instead of rk in $c_{K,1}^*, \dots, c_{K,n}^*$.* In game G_1 , we change the ciphertexts $c_{K,1}^*, \dots, c_{K,n}^*$ containing the wrapped rk to be independent of rk . That is, in Game G_1 we choose a random r with $|rk| = |r|$, and for each recipient uid_i^* of the challenge ciphertext, we replace $c_{K,i}^* = \text{Enc}(upk_i, rk)$ with $c_{K,i}^* = \text{Enc}(upk_i, r)$. To show that this change is indistinguishable, we proceed with a hybrid argument: Let n_t be the number of honest users created by $\mathcal{A}$, i.e. the number of queries to O_{HReg} and let their user identifiers be $(uid_1, \dots, uid_{n_t})$. In each hybrid $G_{1,i}$ with $i \in [n_t]$, we do the following: if user uid_i is one of the recipients of the challenge message, we change the challenge ciphertext for this user to be $c_{K,i}^* = \text{Enc}(upk_i, r)$. Let $G_{1,0} := G_0$. It clearly holds that $G_{1,n_t} = G_1$. We show that each of the game hops from $G_{1,i}$ to $G_{1,i+1}$ are indistinguishable if the PKE_u is IND-CPA secure, thus showing that G_0 and G_1 are indistinguishable.

If uid_{i+1} is not a recipient of the challenge message, then the games are unchanged and it holds that $G_{1,i} = G_{1,i+1}$. Otherwise, assume that there is an efficient distinguisher $\mathcal{D}$ between games $G_{1,i}$ and $G_{1,i+1}$. We can then use this distinguisher to break IND-CPA security of PKE_u . Therefore, let $\mathcal{B}$ be an attacker against PKE_u . For $j \in [i]$ with uid_j is a recipient of the challenge message, $\mathcal{B}$ sets $c_{K,j}^* = \text{Enc}(upk_j, r)$ and for $j = i+2, \dots, n_t$ with uid_j is a recipient of the challenge message, $\mathcal{B}$ sets $c_{K,j}^* = \text{Enc}(upk_j, rk)$. Then, for $j = i+1$, $\mathcal{B}$ embeds the pk received from the IND-CPA experiment of PKE_u as the public key for this recipient, i.e. he sets $upk_j := pk$. It further simulates the decryption oracle $\text{O}_{\text{HDec}}(uid, rid)$ for uid_{i+1}^* by just outputting the previously encrypted message m for rid ($\mathcal{B}$ must know this because the adversary may only decrypt honestly created ciphertexts). Then, it sends messages rk, r to the challenger of PKE_u , and sets the resulting ciphertext as $c_{K,i+1}^*$. We can see that if the challenger encrypts rk , the challenge ciphertexts have been changed to $c_{K,j}^* = \text{Enc}(upk_j, r)$ for all $j \in [i]$ where uid_j is a recipient of the challenge message which corresponds to game $G_{1,i}$, and if the challenger encrypts r , then the challenge ciphertexts have been changed to $c_{K,i}^* = \text{Enc}(upk_i, r)$ for all $j \in [i+1]$ where uid_j is a recipient of the challenge message which corresponds to $G_{1,i+1}$. A successful distinguisher between these games would therefore break IND-CPA security of PKE_u . It therefore holds that $|\text{Adv}_1 - \text{Adv}_0| \leq n_t \cdot \epsilon^{\text{CPA}}$

G_2 : *Encrypt a random message m' instead of m_b in c_D^* .* In game G_2 , we change the challenge ciphertext c_D^* to be an encryption of a random message m' instead of m_b . This change is indistinguishable due to the IND-EAV security of AEAD $_r$. Note that $\mathcal{A}$ has no information about rk anymore from other sources ($c_{K,1}^*, \dots, c_{K,n}^*$ is

independent of rk , and the same rk is never reused.) Therefore, we can construct an adversary $\mathcal{B}$ against IND-EAV security from a distinguisher between games G_1 and G_2 . $\mathcal{B}$ simulates the game exactly as before, but to simulate c_D^* , it queries the IND-EAV challenger on m_b, m' where m' is chosen uniformly random with $|m'| = |m_b|$ and uses the resulting ciphertext as c_D^* . Clearly, if the challenger encrypts m_b , $\mathcal{B}$ simulates game G_1 , and if the challenger encrypts m' , then $\mathcal{B}$ simulates game G_2 perfectly. Any distinguisher between these two games can therefore be used to break IND-EAV security of AEAD. These games are thus indistinguishable, and it holds that $|\text{Adv}_2 - \text{Adv}_1| \leq \epsilon^{\text{EAV}}$

Since G_2 is independent of m_b , the adversary can guess b with a success probability of $1/2$. This concludes the proof, since this means that $\text{Adv}_2 = 1/2$ and therefore $\text{Adv}_0 \leq 1/2 + n_t \cdot \epsilon^{\text{CPA}} + \epsilon^{\text{EAV}}$

D.2 Proof of Theorem 6.2

Proof Idea. The proof consists of a series of game hops which can be clustered into two parts: (i) Making the protocol independent of the password and bounding the adversary to one password guess per interaction with the honest servers, and (ii) replacing the encrypted message and the encrypted keys needed to recover the message with random, ultimately making the game independent of the message m_b . For (i), ultimately we replace the OPRF output $y = H_2(pw, H_1(pw)^{k_{\text{DS}} + k_{\text{KS}}})$ with a random y for all honest users which are recipients of the challenge message. This is indistinguishable if the adversary does not query H_2 exactly on $(pw, H_1(pw)^{k_{\text{DS}} + k_{\text{KS}}})$ through the random oracle assumption. The adversary can further only query q different such inputs after interacting q times with the server via an O_{CDec} or O_{CEnc} query by the security of the 2HashDH building block which is given by the Gap OMDH assumption. This gives the unavoidable bound of $q/|\mathcal{D}_{pw}|$ in the proof. After having replaced y with a random value, the user's signature key-pair is independent of the password and the only way for the adversary to authenticate as the user is to forge a signature of SIG which is not possible due to the unforgeability property of SIG.

Now, in (ii) given that the adversary never manages to authenticate on behalf of an honest user, and can only corrupt one of the servers, he never sees the user's key-related data on *both* servers. We then start replacing the ciphertexts of honest users stored on the key server, i.e. (c_{usk}, c_{dsk}) with random ciphertexts. Notably, the adversary cannot inject ciphertexts for honest users on the key servers. Thus, for $c_{dsk} = \text{Enc}_u(uk, dsk)$ which uses a different uk for each encryption, IND-EAV security suffices, and for $c_{usk} = \text{Enc}_d(dpk, usk)$, IND-CPA security suffices. Next, we want to replace the challenge ciphertext tuples $(c_D^*, c_{K,1}^*, \dots, c_{K,n}^*)$ with random ciphertexts independent of m_b . Since the adversary can inject these ciphertexts when encrypting on behalf of a corrupt user for one of the honest recipients in a O_{CEnc} query, we need to ensure that the decryption (through O_{HDec}) does not leak information about the underlying plaintext. This is ensured due to the one-time ciphertext integrity and the robustness of the AEAD scheme: Whenever the challenge ciphertexts c_D^* or $c_{K,i}^*$ are used for a different rid , the decryption will return $\perp$. Then, we can finally replace c_D^* and $c_{K,1}^*, \dots, c_{K,n}^*$, where IND-EAV security suffices for $c_D^* = \text{AEAD}.\text{Enc}(rk, m_b, rid^*)$

since a fresh rk is used for every encryption, and IND-CCA security yields security of $c_{K,i}^* = \text{Enc}_u(upk, rk)$.

Formal Proof. Let $\mathcal{A}$ be an efficient adversary in the online confidentiality experiment of our π_{DEEP}^+ . Recall that, apart from the honest protocol execution oracles $\mathcal{O}_{\text{HReg}}, \mathcal{O}_{\text{HEnc}}, \mathcal{O}_{\text{HDec}}$, $\mathcal{A}$ can actively participate in the protocol and impersonate users using the $\mathcal{O}_{\text{CReg}}, \mathcal{O}_{\text{CEnc}}$ and $\mathcal{O}_{\text{CDec}}$ oracle, as well as corrupt one of the servers. The goal of $\mathcal{A}$ remains unchanged – determining whether the challenge ciphertexts $c_D^*, c_{K,1}^*, \dots, c_{K,n}^*$ from some honest uid to honest recipients uid^* decrypts to message m_0 or m_1 .

The proof of online confidentiality follows a series of game hops. First, we use a series of game hops (G_1 to G_6) to argue that the attacker can impersonate honest users not better than the optimal bound of one password guess per interaction with the servers. We rely on the Gap OMDH assumption, the random oracle and the unforgeability of the signature scheme to show that these game hops are indistinguishable. Then, we conduct a series of game hops to make the ciphertexts seen by the adversary independent of their real content using the security properties of the encryption schemes. If the key server is not corrupted, these ciphertexts are $(c_D, c_{K,1}, \dots, c_{K,n})$ and if the key server is corrupted, the adversary additionally sees (c_{usk}, c_{dsk}) . Ultimately, we end up with a game which has no information on m_b anymore, and the adversary can only win with probability $1/2$.

Game G_0 . Game G_0 corresponds to the real online unforgeability experiment between $\mathcal{A}$ and the challenger. The challenger simulates the random oracle by lazy sampling of output values, i.e. whenever there is a fresh query to one of the random oracles H_1 or H_2 , the challenger samples a random value from the corresponding output space, and returns this value, and otherwise returns the previously sampled value.

Game G_1 : Abort on random oracle collision. In game G_1 , the challenger aborts on a collision of any of the random oracles, i.e. if he samples a value y on a fresh query x , s.t. y was already output to a different query x' . This ensures that no two passwords can match to the same OPRF output, and is a common game hop in security games. The indistinguishability of this change is given by the birthday bound: $|\text{Adv}_1 - \text{Adv}_0| \leq q_{H_1}^2/2^\lambda + q_{H_2}^2/2^\lambda$ where q_{H_i} is the number of $\mathcal{A}$'s queries to H_i , and 2^λ is the size of the range of both H_1 and H_2 .

Game G_2 : Guess uncorrupted server. In game G_2 , the challenger guesses randomly at the beginning of the game whether DS or KS remains uncorrupted. If the adversary later queries $\mathcal{O}_{\text{Corr}}$ to corrupt the guessed server, the challenger aborts. The games are indistinguishable unless an abort occurs which happens with probability $\leq 1/2$, since the challenger guesses randomly one of the two servers, and in order for the adversary to win, one of the servers must remain uncorrupted. Therefore, $\text{Adv}_2 \geq 1/2 \cdot \text{Adv}_1$.

Game G_3 : Bound number of password guesses. In Game 3, the challenger aborts if there exists an honest user uid_i s.t. there are q_i queries to $\mathcal{O}_{\text{CEnc}}(uid_i)$ and $\mathcal{O}_{\text{CDec}}(uid_i, \cdot)$, but $q_i + 1$ different queries to H_2 of the form (pw, z) s.t. $z = H_1(pw)^{k_{\text{DS}}+k_{\text{KS}}}$ where k_{DS} and k_{KS} are the DS's and KS's keys for uid_i , respectively. Let this

event be called bad_i . This event equates to an adversary evaluating the OPRF on $q_i + 1$ different inputs with only q_i interactions with the servers. Hence, we show that the probability of event bad_i is negligible under the Gap OMDH assumption: Let DS be the uncorrupted server (the case for uncorrupted KS is analogous) and let q_{H_1} be the total number of queries to H_1 in the experiment. The challenger embeds the (q_{H_1}, q) Gap OMDH challenge $(g, g^k, g_1, \dots, g_{q_{H_1}})$ into the experiment as follows: For each new query to $H_1(pw)$, it chooses a fresh $j \in [q_{H_1}]$ and returns g_j . The challenger further simulates $\mathcal{O}_{\text{CEnc}}$ and $\mathcal{O}_{\text{CDec}}$ with the $(\cdot)^k$ oracle to answer the honest DS's response to the adversary's message $\bar{x}$, i.e. it returns $\bar{x}^k$. Other than that, the simulation is identical. Especially, the adversary simulates the KS using a randomly sampled k_{KS} , which he can later also give to the adversary in case of corruption of KS. If the adversary now queries H_2 $q_i + 1$ many times on (pw_j, z_j) s.t. $z_j = H_1(pw_j)^{k_{\text{DS}}+k_{\text{KS}}}$ (which the adversary can detect using the DDH oracle on $(g^k, H_1(pw_j), z_j/H_1(pw_j)^{k_{\text{KS}}})$), then the challenger can output these $q_i + 1$ tuples $(H_1(pw_j), z_j/H_1(pw_j)^{k_{\text{KS}}})$ to win the Gap OMDH game. Since the OPRF keys for all users are chosen independently of each other, it follows that $|\text{Adv}_3 - \text{Adv}_2| \leq n_t \epsilon^{\text{Gap-OMDH}}$.

Game G_4 : Replace blinded values $\bar{x}$. In game 4, for each honest user, the challenger remembers the OPRF output y used to derive the signature key-pair. When simulating the honest encryption and decryption oracles, the challenger sends a random group element $\bar{x} \leftarrow_{\mathcal{R}} \mathbb{G}$ to the servers on behalf of the user, and uses y directly to derive the signature key-pair. Since $\bar{x}$ is a random group element in both games, and the adversary does not see the user's intermediary computations, the games are identical from the adversary's viewpoint, i.e. $\text{Adv}_4 = \text{Adv}_3$.

Game G_5 : Replace y with random. In G_5 , for every honest user uid_i , the value $y = H_2(pw_i, H_1(pw_i)^{k_{\text{DS}}+k_{\text{KS}}})$ used to derive the user's signature key-pair is replaced with a random $y \leftarrow_{\mathcal{R}} \{0, 1\}^\lambda$. Since H_2 is a random oracle, this change is only distinguishable if the adversary queries H_2 exactly on $(pw_i, H_1(pw_i)^{k_{\text{DS}}+k_{\text{KS}}})$ for the user uid_i 's correct password pw_i . Since the adversary has no information anymore about uid_i 's password, he can only guess the password correctly by interacting with the honest server(s) through $\mathcal{O}_{\text{CEnc}}$ and $\mathcal{O}_{\text{CDec}}$ queries and evaluate one password per interaction (see Game 3). Thus, it holds that $|\text{Adv}_5 - \text{Adv}_4| \leq (q_{\mathcal{O}_{\text{CEnc}}} + q_{\mathcal{O}_{\text{CDec}}})/|\mathcal{D}_{pw}|$ where $q_{\mathcal{O}_{\text{CEnc}}}$ and $q_{\mathcal{O}_{\text{CDec}}}$ are the number of queries to $\mathcal{O}_{\text{CEnc}}$ and $\mathcal{O}_{\text{CDec}}$ for all recipient of the challenge ciphertext.

Game G_6 : Abort if attacker authenticates correctly. In game 6, the challenger aborts if $\mathcal{O}_{\text{CEnc}}$ or $\mathcal{O}_{\text{CDec}}$ output succ for an honest user uid . In this case, the attacker has provided valid signatures on the challenge messages chal_{DS} and chal_{KS} . Since the signature keys are independent of the password, and the adversary does not know the secret key sk_s , we can reduce the probability of an abort to the unforgeability of the signature scheme SIG. Thus, the challenger guesses for which user uid the attacker will be able to authenticate correctly, and uses the pk received from the challenger of the unforgeability experiment for SIG as uid 's pk_s . He simulates honest users in the $\mathcal{O}_{\text{HEnc}}$ and $\mathcal{O}_{\text{HDec}}$ oracles with the help of its signing oracle from SIG to produce valid signatures. When the

adversary outputs a valid signature σ_{DS} on “DS”|| $chal_{DS}$ to authenticate, the challenger outputs this as a valid, fresh forgery. The validity is achieved since the attacker authenticates correctly by assumption, and the freshness is ensured since “DS”|| $chal_{DS}$ is chosen freshly by the challenger with high probability. Furthermore, changing the pk is indistinguishable since the adversary never queries H_2 on the correct pw (due to Game 5). It therefore follows that $|\text{Adv}_6 - \text{Adv}_5| \leq \epsilon^{UNF-CMA}$.

Summarizing the Changes. Let us evaluate game G_6 : The adversary in game 6 never authenticates correctly on behalf of an honest user. Thus, the adversary never receives the values stored on both servers to reconstruct the resource key and decrypt the message. What is left to show is that the encryption protects the message, and that the keys which are stored on the KS in encrypted form remain protected as long as the DS is not corrupted.

Game G_7 : Replace c_{dsk} with random. In Game 7, instead of encrypting $c_{dsk} \leftarrow \text{Enc}_u(uk, dsk)$, the challenger encrypts $c_{dsk} \leftarrow \text{Enc}_u(uk, r)$ for a random value r with $|r| = |dsk|$ for each honest user during registration. The challenger uses dsk directly (instead of decrypting c_{dsk}) to simulate $\mathcal{O}_{HEnc}$ and $\mathcal{O}_{HDec}$ queries. The games are indistinguishable if SE is IND-EAV secure. Note that uk is only used once for encryption, that the attacker never sees the honestly decrypted dsk as it is an internal value of the user and it cannot inject a malicious c_{dsk} . There are two cases: (i) the attacker does not corrupt KS. Then the attacker never sees c_{dsk} (as he also does not get it by impersonating the honest user due to the previous game hops). In this case, the games are clearly indistinguishable. (ii) the attacker corrupts KS. In this case, he sees c_{dsk} but not the key uk because he does not impersonate the user correctly (again previous game hop), and DS is not corrupted (otherwise both servers would be corrupted). In this case, we can make a straightforward reduction to the IND-EAV security of SE. Thus, $|\text{Adv}_7 - \text{Adv}_6| \leq n_t \cdot \epsilon^{IND-EAV}$.

Game G_8 : Replace c_{usk} with random. In Game 8, instead of encrypting $c_{usk} \leftarrow \text{Enc}_d(dpk, usk)$, the challenger encrypts $c_{usk} \leftarrow \text{Enc}_d(dpk, r)$ for a random value r with $|r| = |usk|$ for each honest user during registration. The challenger uses usk directly (instead of decrypting c_{usk}) to simulate $\mathcal{O}_{HEnc}$ and $\mathcal{O}_{HDec}$ queries. The games are indistinguishable if PKE_d is IND-CPA secure. Again, the attacker never sees usk because it is an internal value of the user, and the attacker cannot influence c_{usk} , thus there is no decryption oracle for c_{usk} . Again, there are two cases: (i) if the attacker never corrupts KS, he does not see c_{usk} and indistinguishability is trivial; (ii) if the attacker corrupts KS, he sees c_{usk} . In this case, he also knows dpk and can therefore encrypt arbitrary messages (thus we need CPA-security) but he has no information about dsk (due to the previous game hop). Hence, we can make a reduction to IND-CPA security of PKE_d , and it holds that $|\text{Adv}_8 - \text{Adv}_7| \leq n_t \cdot \epsilon^{IND-CPA}$.

Game G_9 : Change simulation of $\mathcal{O}_{HDec}$. In Game 9, for every $uid \in \{uid_i^*\}_{i=1}^n$, whenever there is a query to $\mathcal{O}_{HDec}(uid, rid)$ such that $(rid, c_D^*) \in DB_D$ or $(rid, uid, c_{K,i}^*) \in DB_K$, i.e. one of the challenge messages c_D^* or $c_{K,i}^*$ for the recipient user uid is used for a different $rid \neq rid^*$, the challenger simulates the output of the decryption protocol to be $\perp$. This can happen if the $\mathcal{O}_{HEnc}$ oracle returns one of the challenge ciphertexts for a different rid or if

the adversary queries $\mathcal{O}_{CEnc}$ and during the encryption protocol, sends c_D^* to DS or $c_{K,i}^*$ to KS as ciphertext for uid . This game hop is indistinguishable unless there is a tuple $(c_D, c_{K,i}, rid)$ with $rid \neq rid^* \wedge (c_D = c_D^* \vee c_{K,i} = c_{K,i}^*)$ s.t. $\mathcal{O}_{CDec}$ decrypts to a valid $m \neq \perp$ for the honest recipient uid . We show that this cannot happen due to the one-time ciphertext integrity and the robustness of AEAD. Therefore, we consider three cases for $(c_D, c_{K,i}, rid)$ which may decrypt to a valid $m \neq \perp$:

- (1) $c_D = c_D^* \wedge c_{K,i} = c_{K,i}^*$: In this case, it must hold that $\text{AEAD.Dec}(rk, c_D^*, rid^*) \neq \perp$ and $\text{AEAD.Dec}(rk, c_D^*, rid) \neq \perp$ for the same rk (since $c_{K,i} = c_{K,i}^*$) and different rid 's. This clearly violates the one-time ciphertext integrity of AEAD, and we can make a straightforward reduction to the one-time ciphertext integrity game of AEAD.
- (2) $c_D \neq c_D^* \wedge c_{K,i} = c_{K,i}^*$: This is similar to case 1, as it must hold that $\text{AEAD.Dec}(rk, c_D^*, rid^*) \neq \perp$ and $\text{AEAD.Dec}(rk, c_D, rid) \neq \perp$ for the same rk (since $c_{K,i} = c_{K,i}^*$) which again violates the one-time ciphertext integrity of AEAD.
- (3) $c_D = c_D^* \wedge c_{K,i} \neq c_{K,i}^*$: In this case, the adversary would break the robustness of the scheme, as it would hold that c_D^* decrypts properly under two distinct pairs (rk^*, rid^*) and (rk, rid) . We can also show this with a straightforward reduction to the robustness property of AEAD.

Game G_{10} : Replace $c_{K,i}$ with random: In Game 10, we change the ciphertexts $c_{K,i}$ to be independent of the resource key rk . This game hop is almost identical to the game hop of G_1 in the strong confidentiality case for Doctolib's DEEP, except that we also have to account for maliciously generated ciphertexts, and use the decryption oracle of the CCA-challenger for PKE_u to simulate the $\mathcal{O}_{HDec}$ oracle. Since we have ruled out that the challenge ciphertext is ever used in one of these decryption oracles, the attacker in the CCA experiment never queries the challenge ciphertext and the reduction is solid.

Game G_{11} : Replace c_D with random: In game G_{11} , the challenge ciphertext c_D^* is changed to be independent of m_b , i.e. the challenger encrypts a random message m . Since the attacker has no information on rk (Game 10), and rk is chosen randomly for each encryption, the attacker has no encryption oracle. Furthermore, since c_K^* is never re-used in one of the $\mathcal{O}_{HDec}$ queries, the attacker never sees a decryption under the same rk . Thus, there is also not a decryption oracle available to the attacker, and the games are indistinguishable due to the IND-EAV security of AEAD.

Since now in Game 11, there is no information about m_b anymore, the attacker can only guess and the winning probability is 1/2. This concludes the proof.

D.3 Proof of Theorem 6.3

Proof. In the precomputation attack resistance experiment, the adversary tries to recover the password of an honest user. The adversary may compromise a server, in which case, he receives pk_S , the server's OPRF key (k_{DS} or k_{KS}) and some user-key related values which are independent of the password. Only pk_S is derived deterministically from pw as $pk_S = \text{KGen}(1^\lambda; y)$ with $y = H_2(pw, H_1(pw)^{k_{DS}+k_{KS}})$. The adversary cannot precompute these y values before compromising both servers, since the key of

the uncompromised server has high-entropy. After compromising both servers, the adversary additionally has to query H_2 on the correct $(pw, H_1(pw)^{k_{DS}+k_{KS}})$ in order to learn the password due to the RO-assumption. Thus, the adversary can only learn the password by querying H_2 on the correct password *after* compromising

both servers. We count each query to H_2 on a tuple of the form $(pw, H_1(pw)^{k_{DS}+k_{KS}})$ as a query to O_{TestPwd} , which concludes the proof.